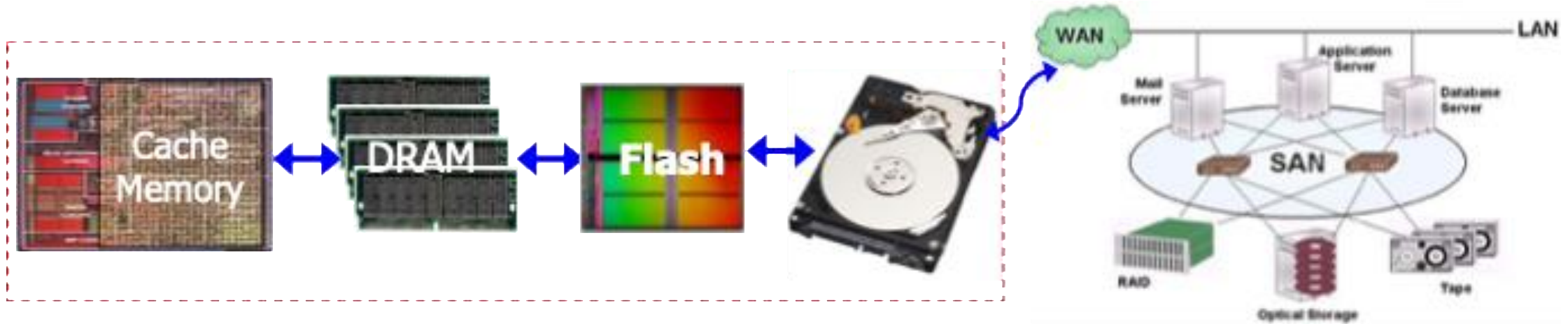


ECSE 4320/6320 Advanced Computer Systems

Thread 1: How to Speed Up Computing? Part 1

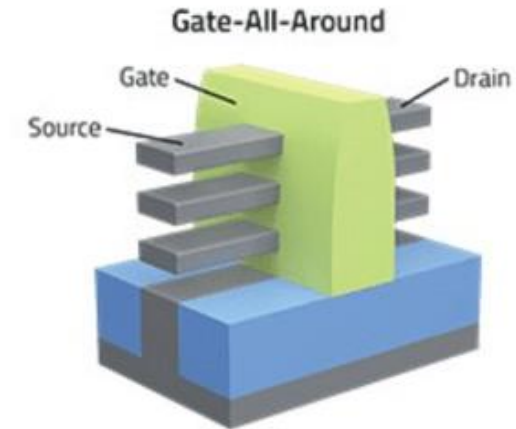
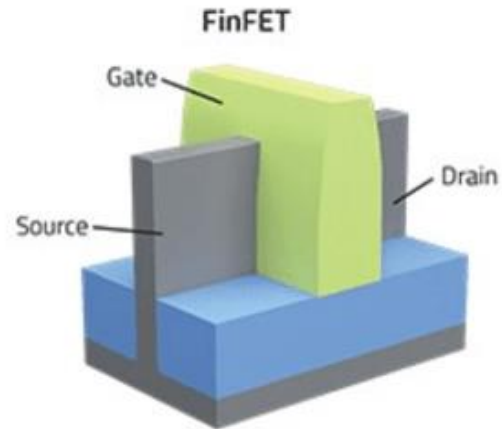
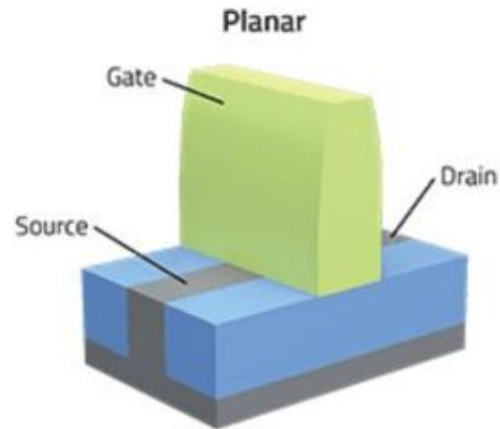
Speed Up Computing



Device ↔ Microarchitecture ↔ Architecture ↔ Infrastructure SW ↔ Application SW

Device Innovation

Move electrons faster

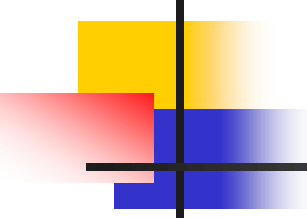


PCI
EXPRESS®

High-speed links

NVIDIA
NVLINK





Speed Up Computing

Device ↔ Microarchitecture ↔ Architecture ↔ Infrastructure SW ↔ Application SW

□ Let's now start from microarchitecture

- Microarchitecture (μarch) is the internal design of a processor that implements an ISA, i.e., how the chip actually executes the ISA's instructions
- **Pipelines & execution:** decode, micro-ops, out-of-order scheduling, issue width, ALUs/FMAs, vector units
- **Speculation:** branch predictors, speculative execution, register renaming, reorder buffer
- **Caches & memory:** L1/L2/L3 sizes/latencies, prefetchers, memory controllers, interconnects

What are the low-hanging fruits here?

CANOS Recap

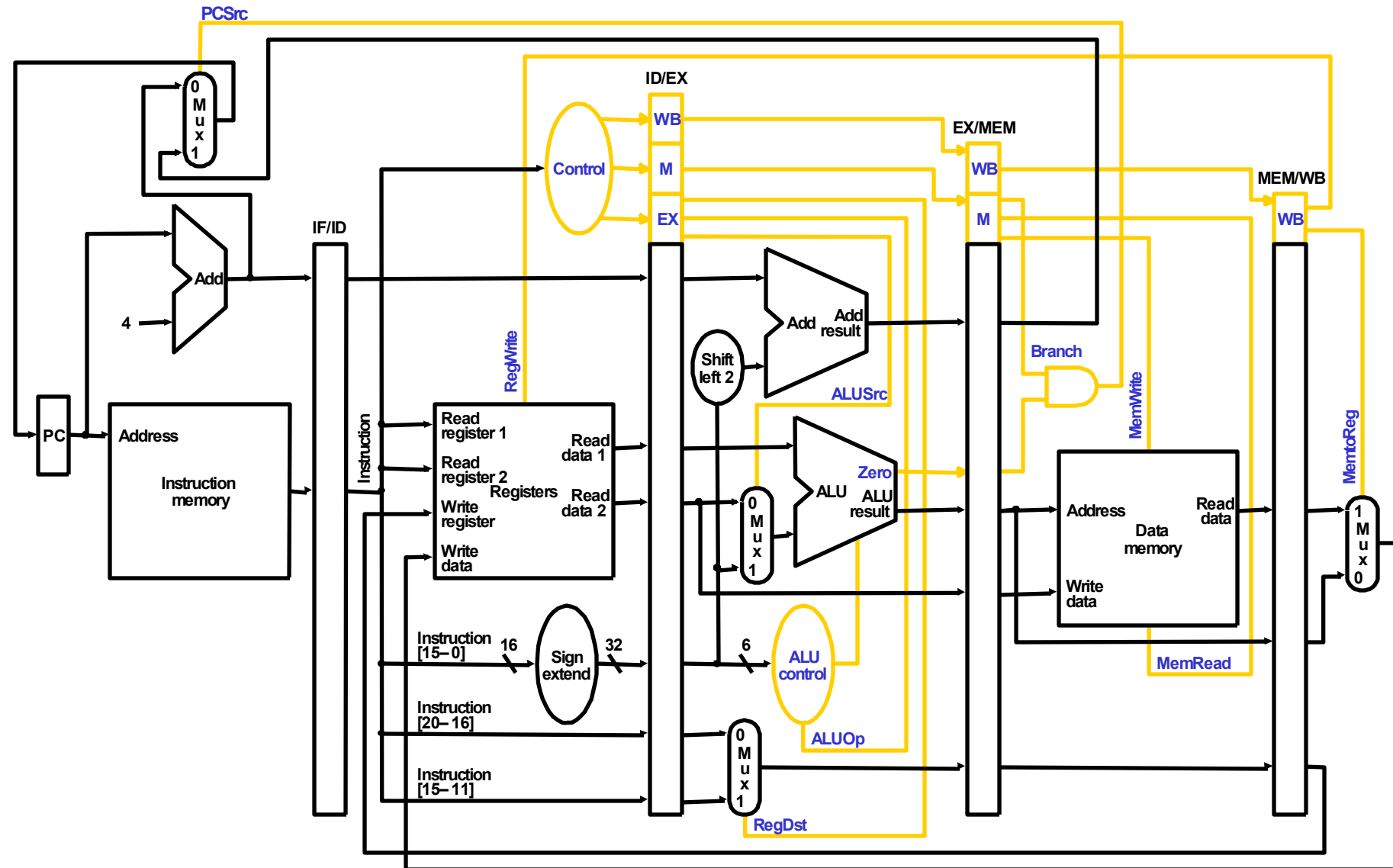
Pipelined data path

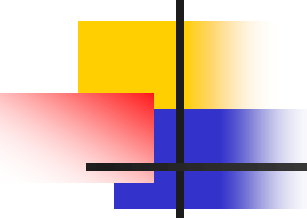


Higher instruction level
parallelism (ILP)



Higher speed





CANOS Recap

Instruction level parallelism (ILP)  Inter-instruction data dependency

❑ **Data Hazard:** Attempt to use an item before it is ready (i.e., instruction depends on result of prior instruction still in the pipeline)

➤ Register forwarding

❑ **Control Hazard:** Attempt to make a decision before condition is evaluated

➤ Branch prediction

❑ **Structural Hazards:** Contention for same hardware resource.

Computing

Latency-Oriented Computing



Minimize the time it takes to perform a single task

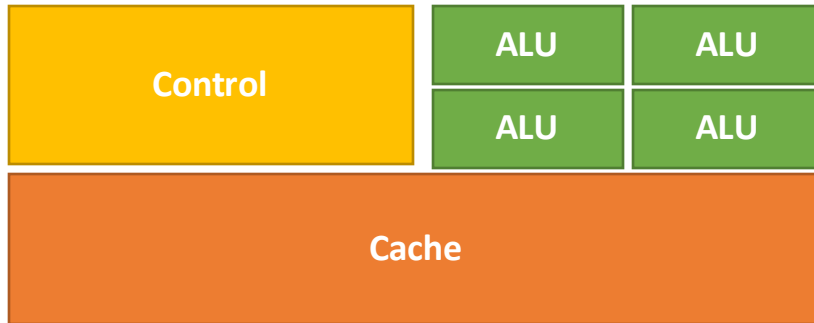
Throughput-Oriented Computing



Maximize the number of tasks that can be performed in a given time frame

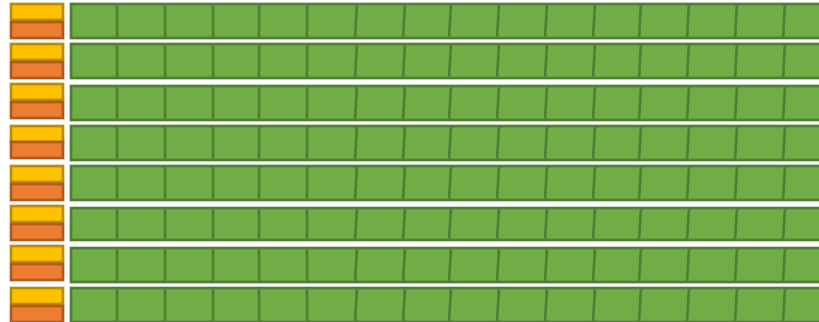
Computing

CPU: Latency-Oriented Computing



- A few powerful **ALUs**
 - Reduced operation latency
- Large **caches**
 - Convert long latency memory accesses to short latency cache accesses
- Sophisticated **control**
 - Branch prediction to reduce control hazards
 - Data forwarding to reduce data hazards
- Modest multithreading to hide short latency
- High clock frequency

GPU: Throughput-Oriented Computing



- Many small **ALUs**
 - Long latency, high throughput
 - Heavily pipelined for further throughput
- Small **caches**
 - More area dedicated to computation
- Simple **control**
 - More area dedicated to computation
- Massive number of threads to hide very high latency!
- Moderate clock frequency

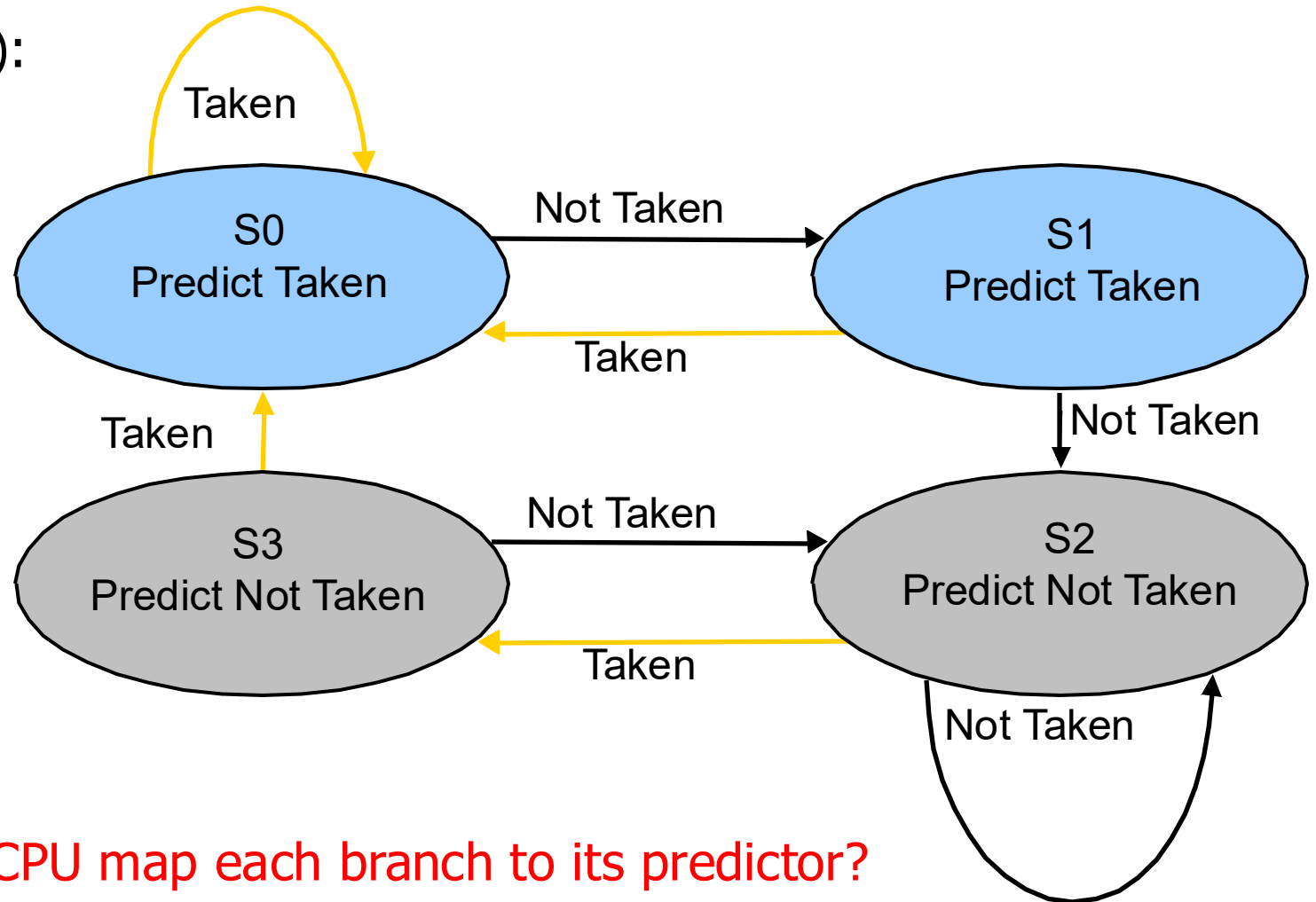
Branch Prediction

❑ Basic 2-bit predictor (from CANOS):

➤ For each branch:

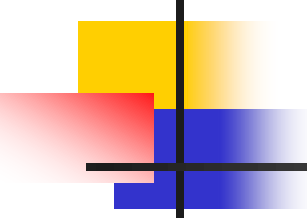
❖ Predict taken or not taken

❖ If the prediction is wrong two consecutive times, change prediction



Practical implementation? How does CPU map each branch to its predictor?

How could we further improve the prediction accuracy?



Branch Prediction

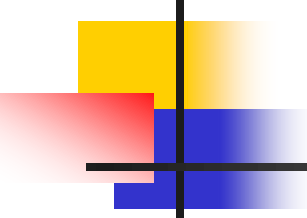
❑ Local predictor:

- Multiple 2-bit predictors for each branch
- One for each possible combination of outcomes for the last n occurrences of this branch

❑ Correlating predictor:

- Multiple 2-bit predictors for each branch
- One for each possible combination of outcomes of preceding n branches

As pipelining is getting deeper, how would it interact with branch prediction?

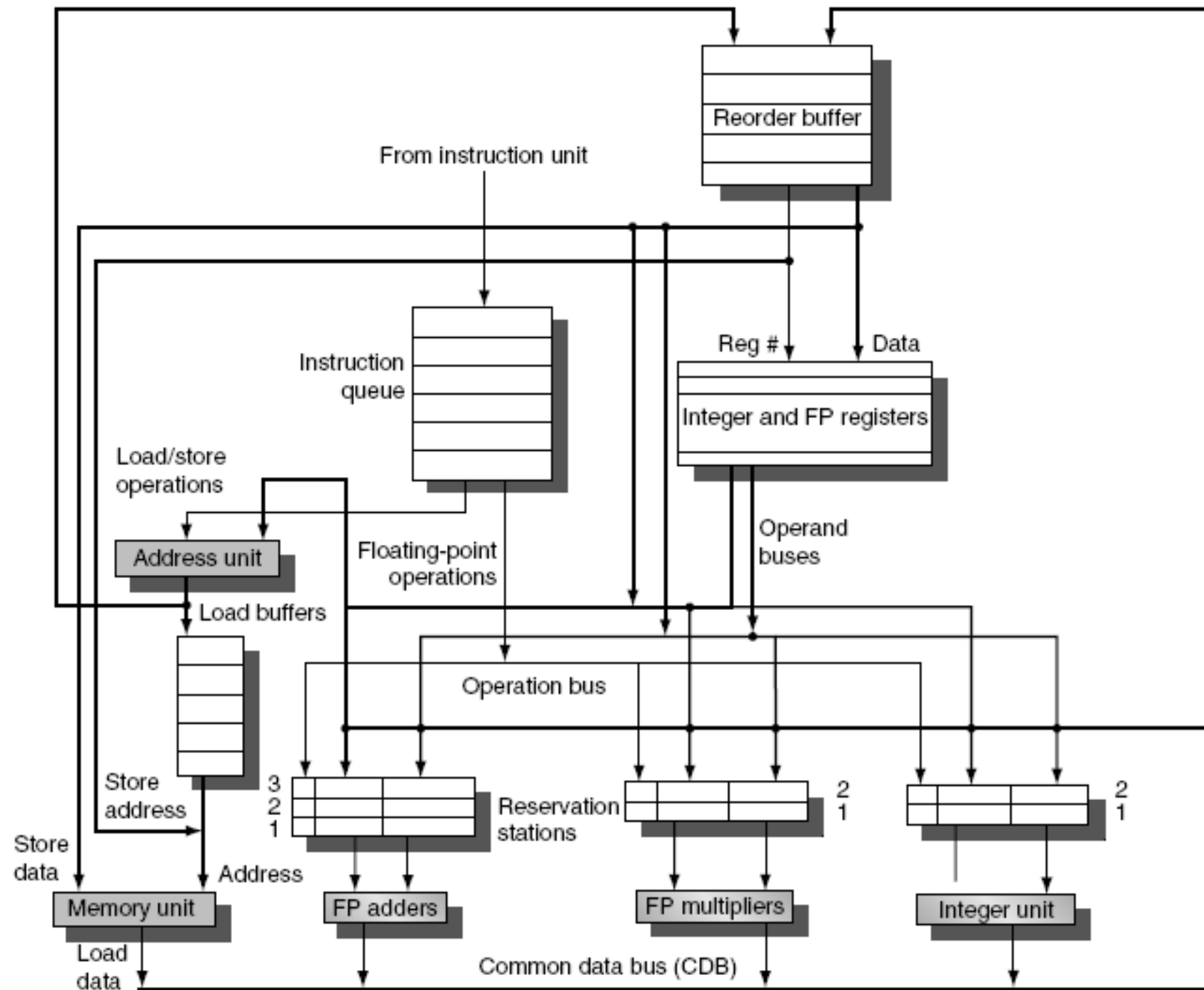
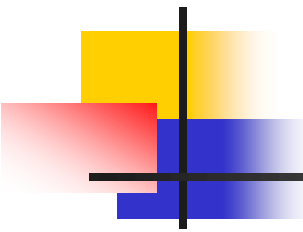


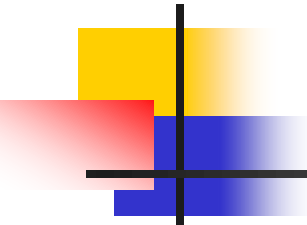
Hardware-Based Speculation

- ❑ Aggressive look-ahead: Execute instructions along predicted execution paths but only commit the results if prediction was correct
- ❑ Instruction commit: Allow an instruction to update the register file when instruction is no longer speculative
- ❑ Need additional hardware to prevent any irrevocable action until an instruction commits



- ✓ Reorder buffer: holds the result of instruction between completion and commit
 - Register values and memory values are not written until an instruction commits
 - Speculated entries in ROB are cleared in the case of misprediction





Dynamic Scheduling

Instruction level parallelism (ILP)  Inter-instruction data dependency

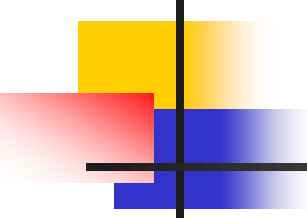
- ❑ Rearrange the order of instructions to reduce stalls while maintaining data flow

- ❑ Advantages:

- Compiler doesn't need to have knowledge of microarchitecture
- Handles cases where dependencies are unknown at compile time

- ❑ Disadvantage:

- **Substantial** increase in hardware complexity
- Complicates exceptions and corner cases



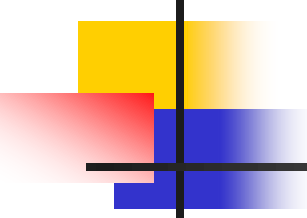
Dynamic Scheduling

- ❑ Dynamic scheduling implies:
 - **Out-of-order** execution
 - **Out-of-order** completion
- ❑ WAR (write-after-read) and WAW (write-after-write)

add R1, R2, R3

sub R2, R4, R1

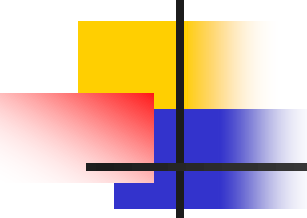
add R1, R6, R3



Dynamic Scheduling

❑ Tomasulo's Approach

- Tracks when operands are available
- Introduces register renaming in hardware
 - ❖ Minimizes WAW and WAR hazards



Register Renaming

❑ Example:

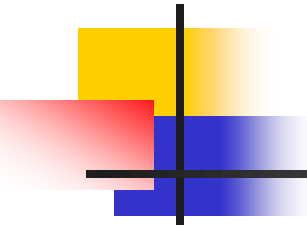
DIV.D F0, F2, F4

ADD.D F6, F0, F8

S.D F6, 0(R1)

SUB.D F8, F10, F14

MUL.D F6, F10, F8



Register Renaming

❑ Example:

DIV.D F0, F2, F4

ADD.D F6, F0, F8

S.D F6, 0(R1)

SUB.D F8, F10, F14

MUL.D F6, F10, F8



DIV.D F0, F2, F4

ADD.D **S**, F0, F8

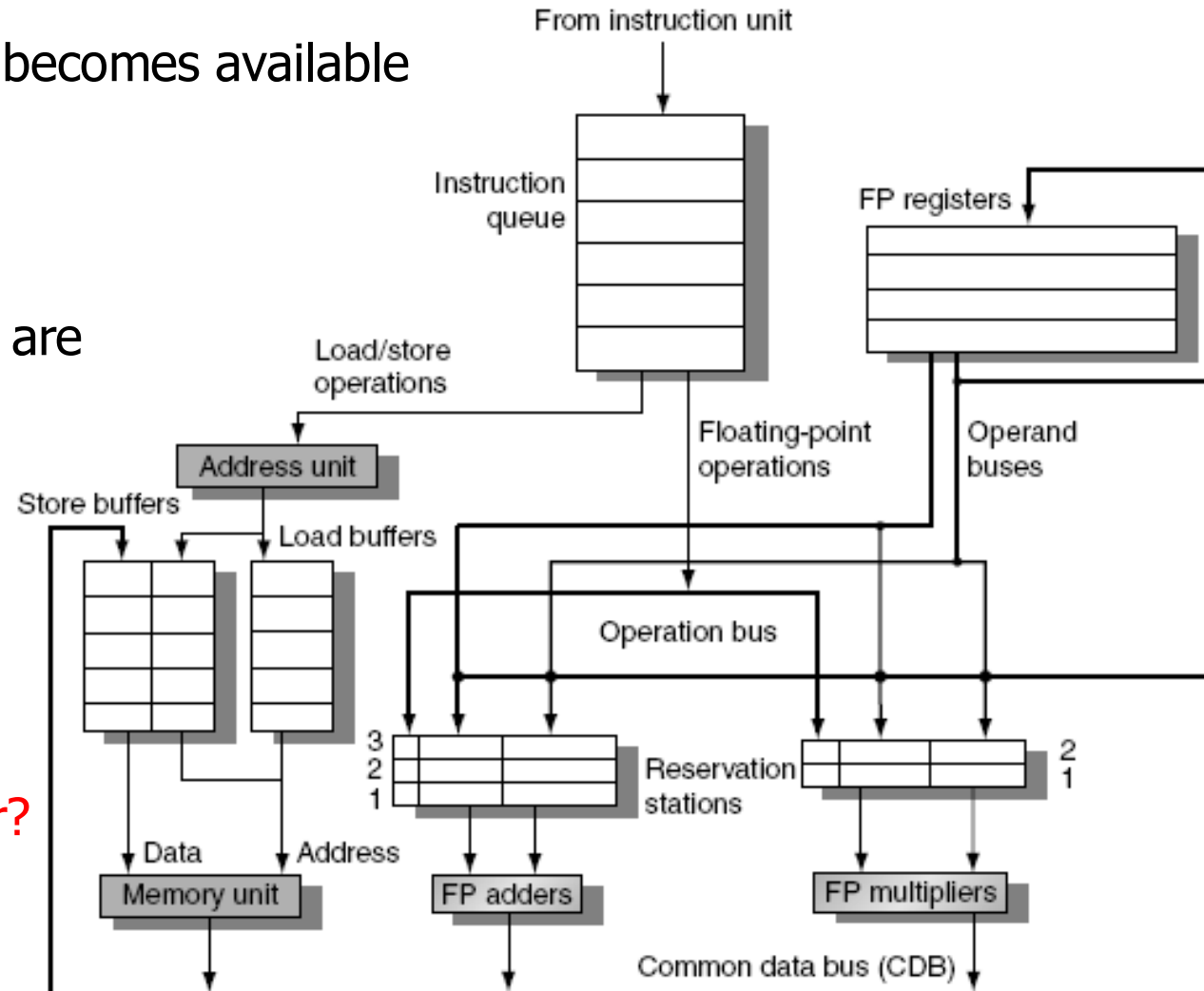
S.D **S**, 0(R1)

SUB.D **T**, F10, F14

MUL.D F6, F10, **T**

Register Renaming

- ❑ Register renaming is enabled by reservation stations (RS), aka unified scheduler
- ❑ RS fetches and buffers an operand as soon as it becomes available (not necessarily coming from register file)
- ❑ Only the last output updates the register file
- ❑ As instructions are issued, the register specifiers are renamed with the reservation station
- ❑ Maybe more reservation stations than registers
- ❑ Load and store buffers: Contain data and addresses, act like reservation stations

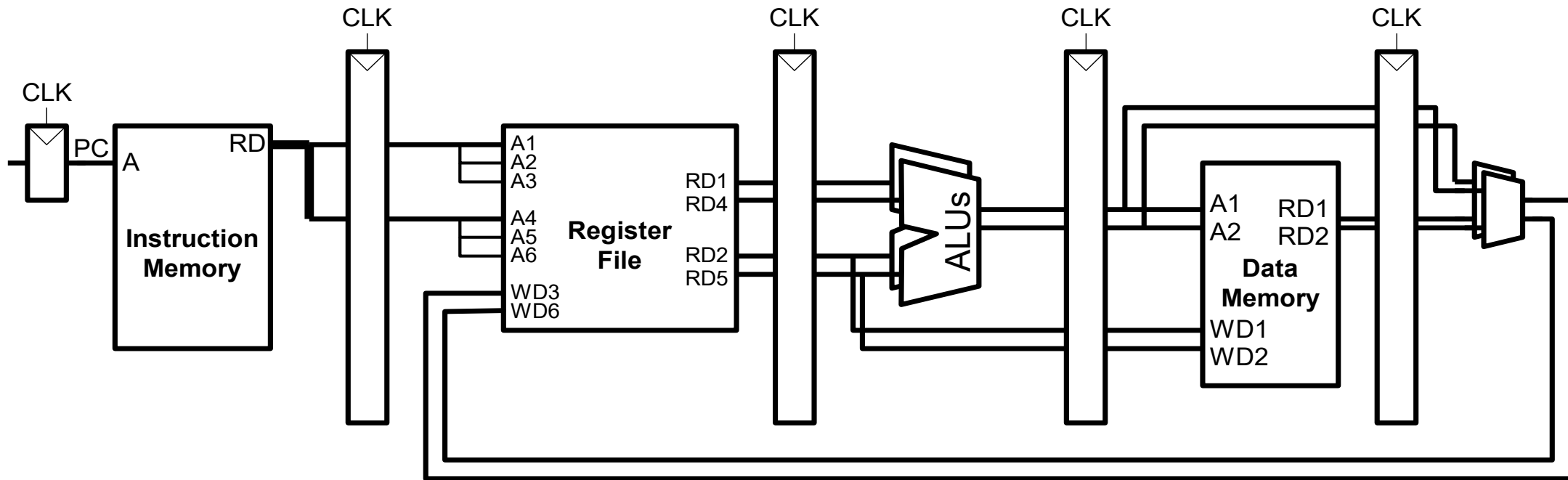


Why not simply exposing more registers to compiler?

Any other options to improve ILP?

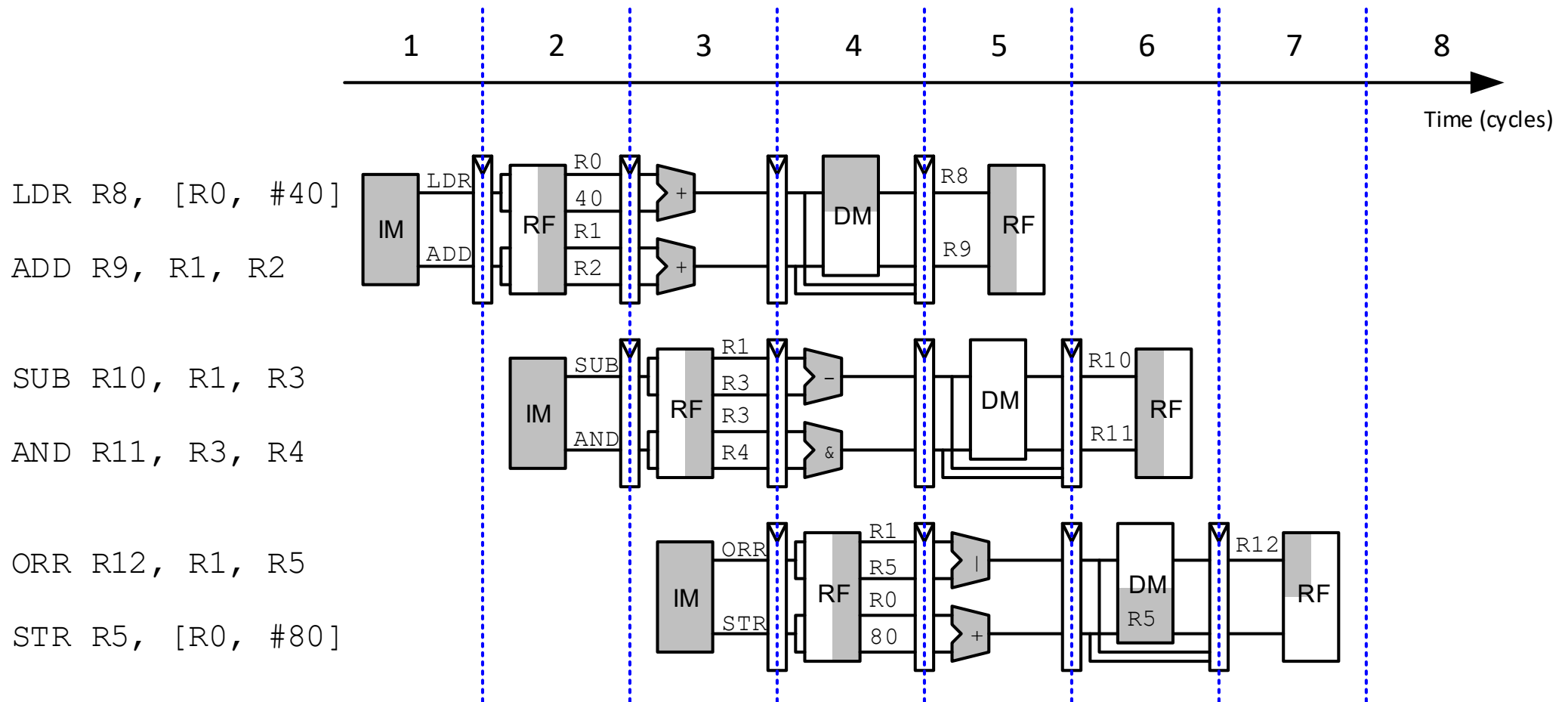
Superscalar

- ❑ Multiple copies of datapath execute multiple instructions at once
- ❑ Dependencies make it tricky to issue multiple instructions at once



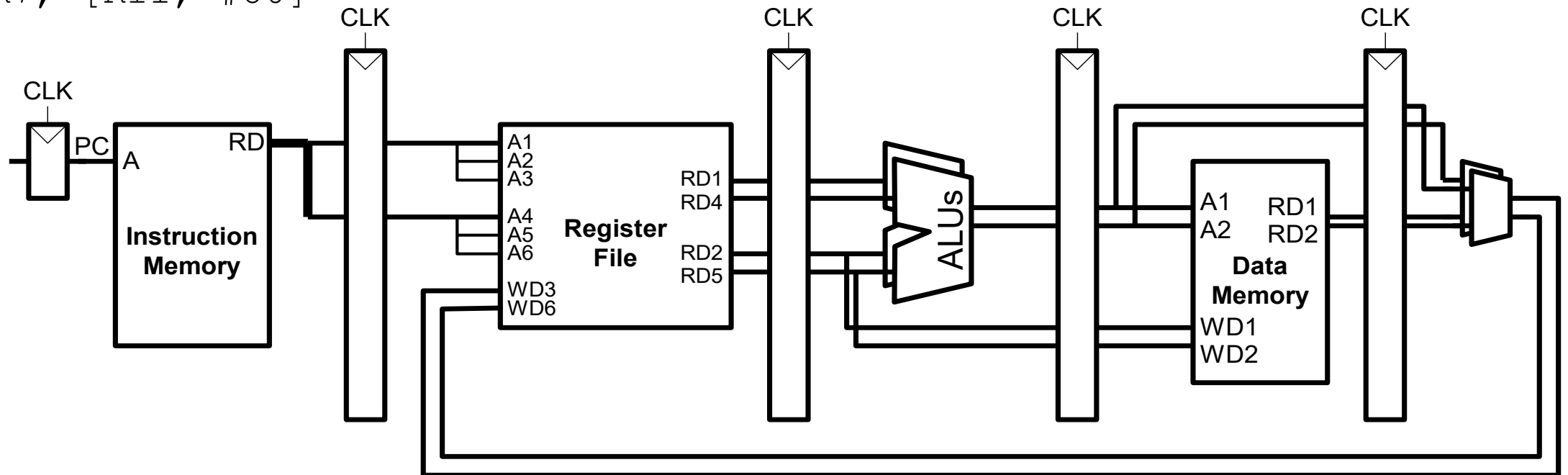
Superscalar

Ideal IPC: 2 & Actual IPC: 2



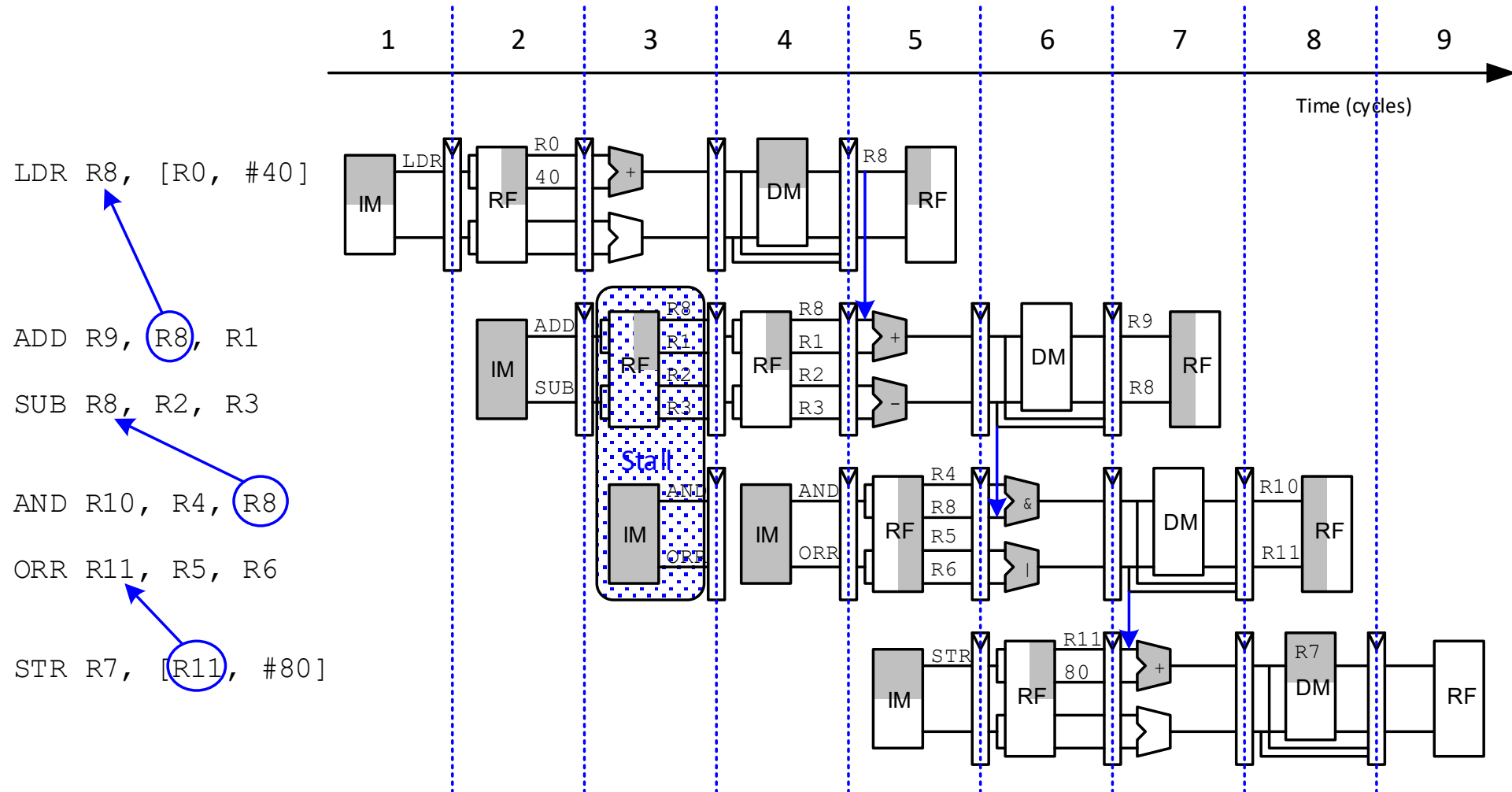
Superscalar

```
LDR  R8, [R0, #40]
ADD  R9, R8, R1
SUB  R8, R2, R3
AND  R10, R4, R8
ORR  R11, R5, R6
STR  R7, [R11, #80]
```



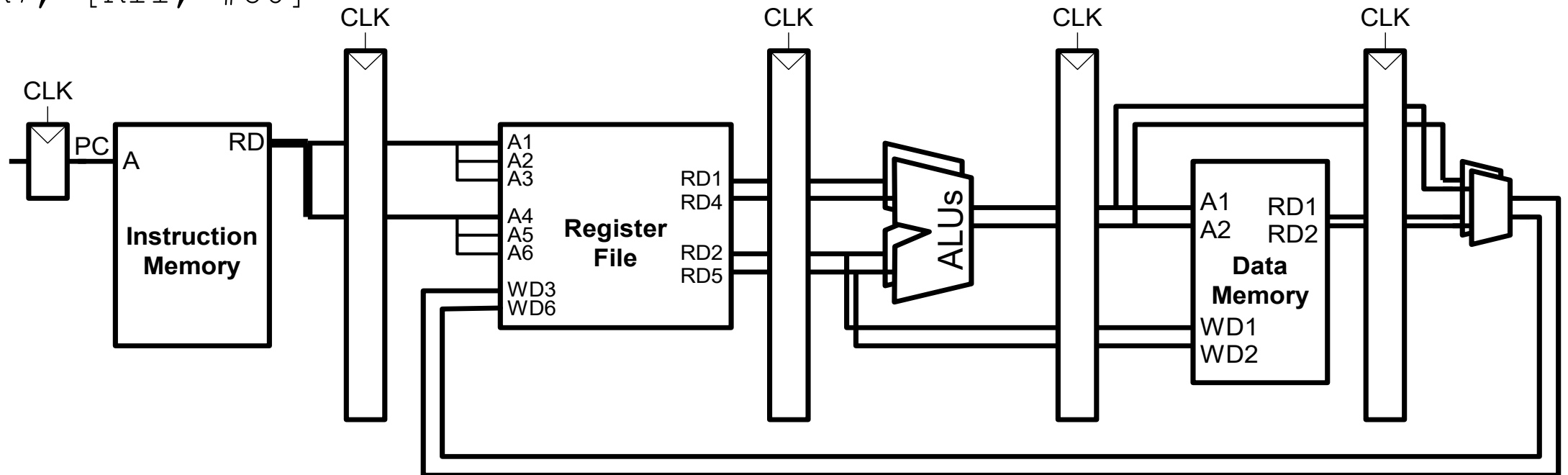
Superscalar

Ideal IPC: 2 & Actual IPC: $6/5=1.2$



Superscalar

```
LDR  R8, [R0, #40]
ADD  R9,  R8, R1
SUB  R8,  R2, R3
AND  R10, R4, R8
ORR  R11, R5, R6
STR  R7, [R11, #80]
```



Superscalar

LDR R8, [R0, #40]

ADD R9, R8, R1

SUB R8, R2, R3

AND R10, R4, R8

ORR R11, R5, R6

STR R7, [R11, #80]

Ideal IPC: 2 & Actual IPC: $6/4=1.5$

LDR R8, [R0, #40]

ORR R11, R5, R6

STR R7, [R11, #80]

ADD R9, R8, R1

SUB R8, R2, R3

AND R10, R4, R8

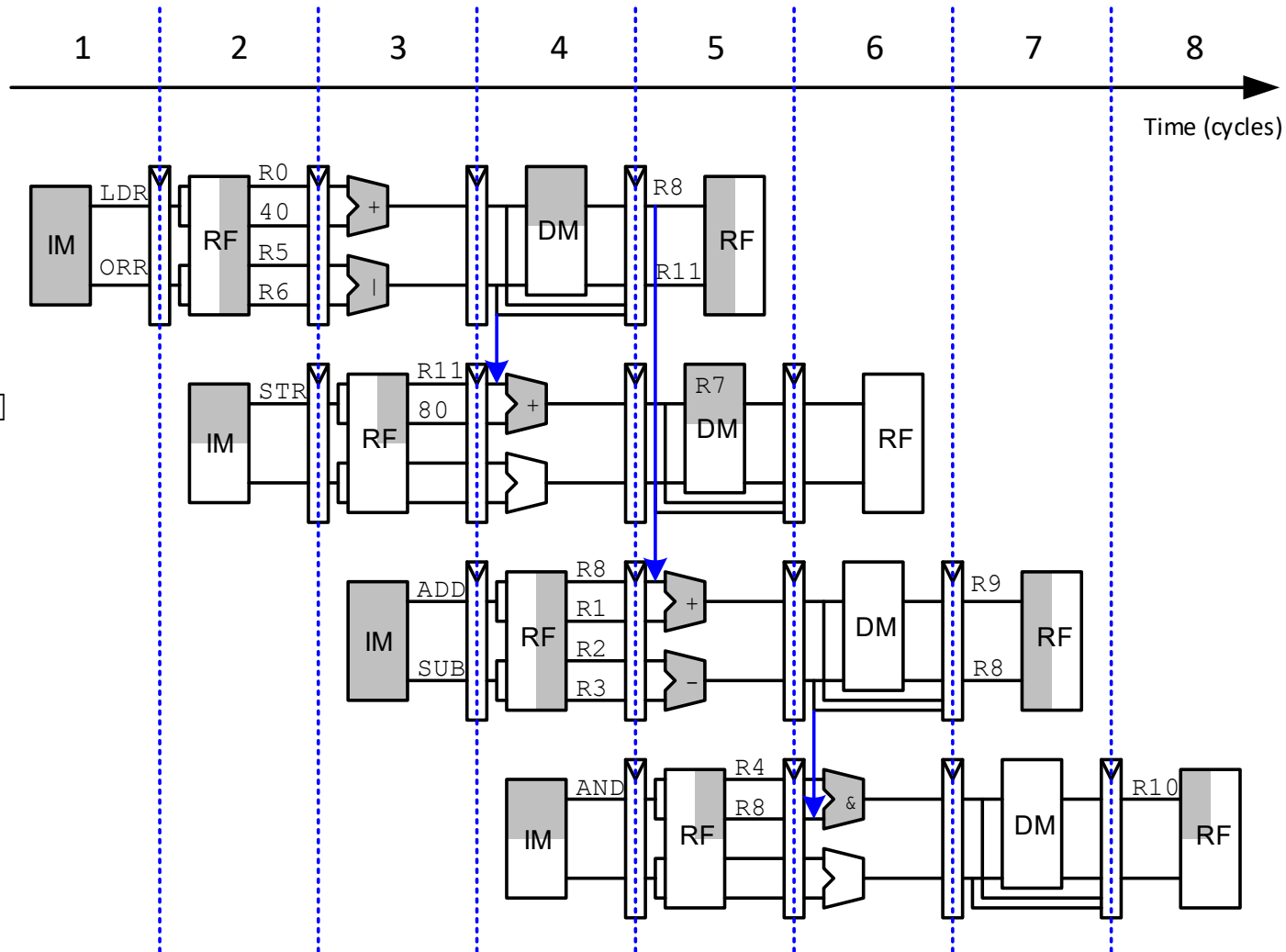
two cycle latency
between load and
use of R8

RAW

RAW

WAR

RAW



Superscalar

LDR R8, [R0, #40]

ADD R9, R8, R1

SUB R8, R2, R3

AND R10, R4, R8

ORR R11, R5, R6

STR R7, [R11, #80] LDR R8, [R0, #40]

SUB T0, R2, R3

AND R10, R4, T0

ORR R11, R5, R6

ADD R9, R8, R1

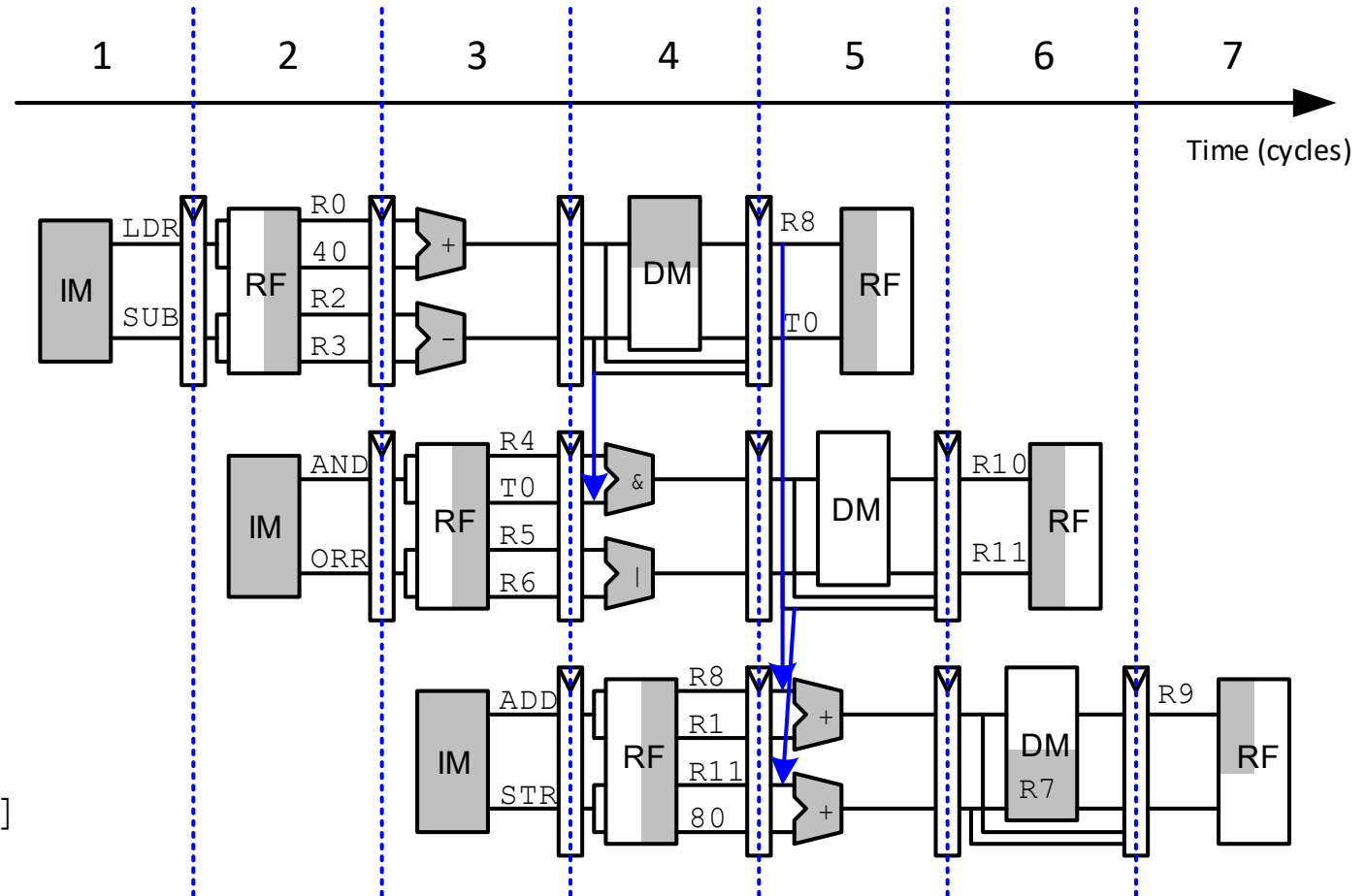
STR R7, [R11, #80]

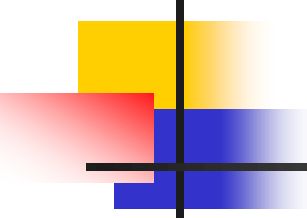
2-cycle RAW

RAW

RAW

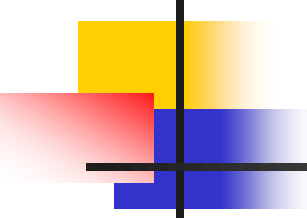
Ideal IPC: 2 & Actual IPC: $6/3=2$





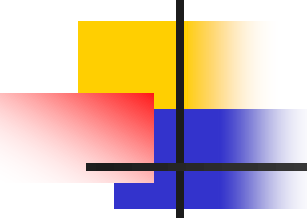
State of the Art

- ❑ Modern CPU microarchitectures
 - Dynamic scheduling (out-of-order) + multiple issue (superscalar) + speculation
 - Deep pipelining (e.g., 10~20 stages)
- ❑ Issue logic can become bottleneck
 - Limit the number of instructions of a given class that can be issued in a “bundle” (e.g., one FP, one integer, one load, one store)
- ❑ A VLIW (Very Long Instruction Word) processor fetches and executes a single, wide instruction containing multiple independent operations in parallel, with the compiler deciding the scheduling instead of hardware
 - Intel began working on VLIW-based Itanium CPU in the early 1990s in partnership with HP and released the first Itanium CPU in 2001, but officially ended Itanium CPU shipment in 2020
 - **Why? What lessons could we learn?**



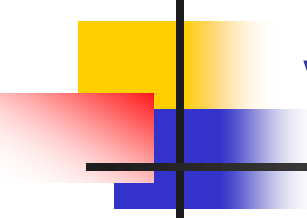
Data-level Parallelism

- ❑ SIMD (single instruction multiple data) architectures can exploit data-level parallelism for:
 - matrix-oriented scientific computing
 - media-oriented image and sound processors
- ❑ SIMD is more energy efficient than MIMD
 - Only needs to fetch one instruction per data operation
 - Makes SIMD attractive for personal mobile devices
- ❑ SIMD allows programmer to continue to think sequentially



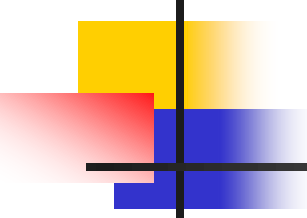
SIMD Parallelism

- ☐ Vector architectures
- ☐ SIMD extensions
- ☐ Graphics Processor Units (GPUs)



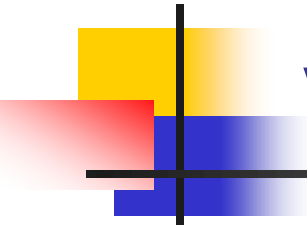
Vector Architectures

- ❑ Basic idea:
 - Read sets of data elements into “vector registers”
 - Operate on those registers
 - Disperse the results back into memory
- ❑ Registers are controlled by compiler
 - Used to hide memory latency
 - Leverage memory bandwidth



Example: VMIPS

- ❑ Loosely based on Cray-1
- ❑ Vector registers
 - Each register holds a 64-element, 64 bits/element vector
 - Register file has 16 read ports and 8 write ports
- ❑ Vector functional units: Fully pipelined
 - Data and control hazards are detected
- ❑ Vector load-store unit: Fully pipelined
 - One word per clock cycle after initial latency
- ❑ Scalar registers
 - 32 general-purpose registers
 - 32 floating-point registers



VMIPS Instructions

- ❑ ADDVV.D: add two vectors
- ❑ ADDVS.D: add vector to a scalar
- ❑ LV/SV: vector load and vector store from address

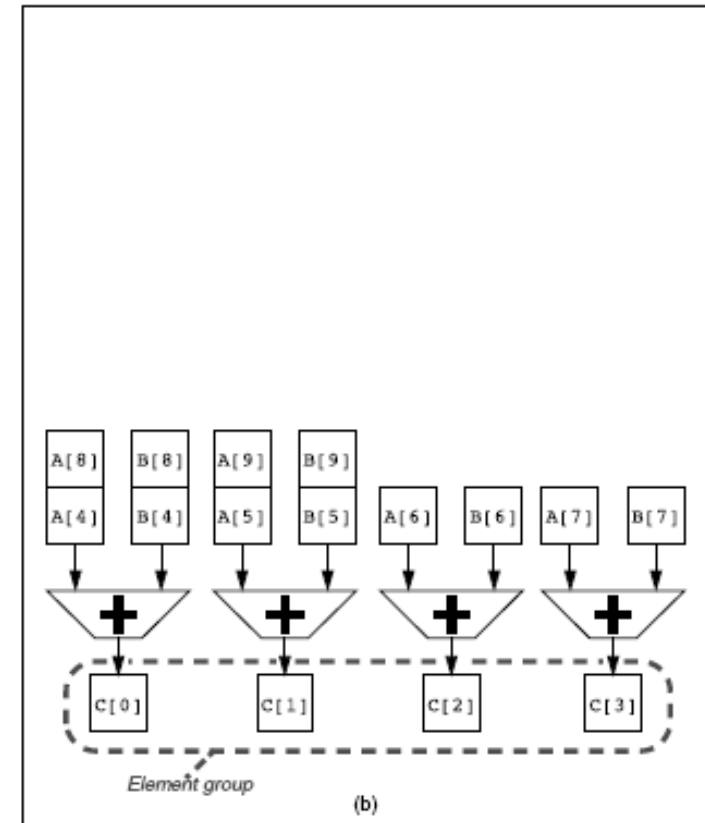
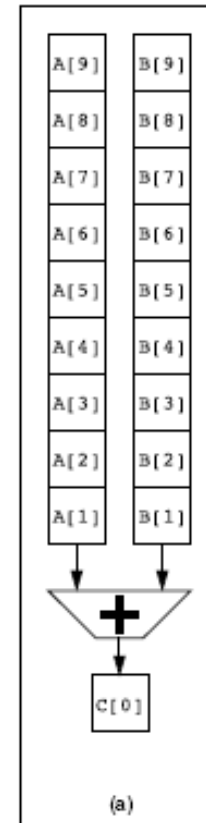
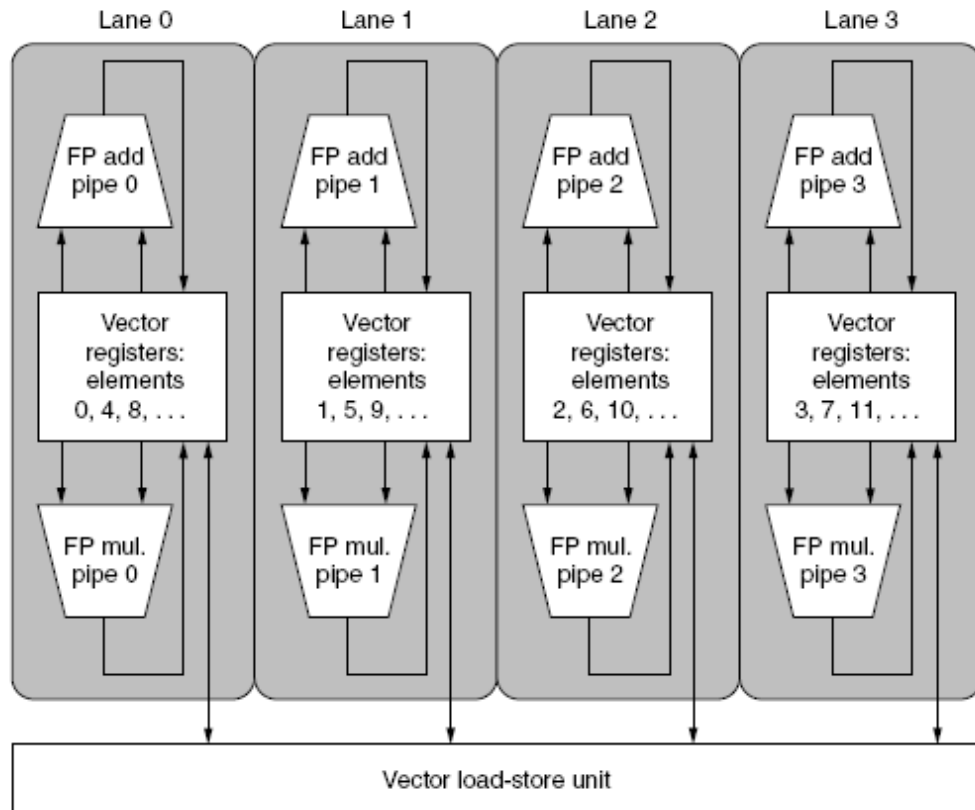
- ❑ Example:

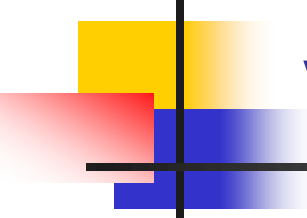
L.D	F0, a	; load scalar a
LV	V1, Rx	; load vector X
MULVS.D	V2, V1, F0	; vector-scalar multiply
LV	V3, Ry	; load vector Y
ADDVV	V4, V2, V3	; add
SV	Ry, V4	; store the result

- ❑ Requires 6 instructions vs. almost 600 for MIPS

Vector Execution Time

- ❑ Execution time depends on three factors:
 - Length of operand vectors, Structural hazards, Data dependencies
- ❑ VMIPS functional units consume one element per clock cycle → Execution time is approximately the vector length → How to further improve performance?





Vector Mask Registers

❑ Consider:

for ($i = 0$; $i < 64$; $i=i+1$)

if ($X[i] \neq 0$)

$X[i] = X[i] - Y[i]$;

❑ Use vector mask register to “disable” elements:

LV	V1,Rx	;load vector X into V1
----	-------	------------------------

LV	V2,Ry	;load vector Y
----	-------	----------------

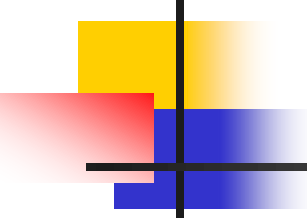
L.D	F0,#0	;load FP zero into F0
-----	-------	-----------------------

SNEVS.D	V1,F0	;sets VM(i) to 1 if V1(i)≠F0
---------	-------	------------------------------

SUBVV.D	V1,V1,V2	;subtract under vector mask
---------	----------	-----------------------------

SV	Rx,V1	;store the result in X
----	-------	------------------------

❑ GFLOPS rate decreases!



Scatter-Gather

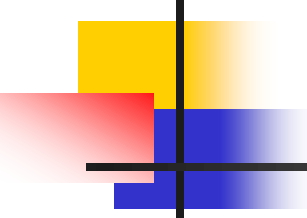
❑ Consider:

for ($i = 0$; $i < n$; $i=i+1$)

$A[K[i]] = A[K[i]] + C[M[i]]$;

❑ Use index vector:

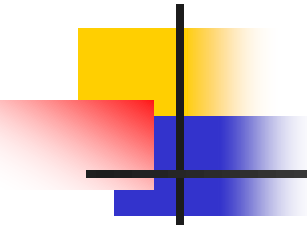
LV	Vk, Rk	;load K
LVI	Va, (Ra+Vk)	;load A[K[]]
LV	Vm, Rm	;load M
LVI	Vc, (Rc+Vm)	;load C[M[]]
ADDVV.D	Va, Va, Vc	;add them
SVI	(Ra+Vk), Va	;store A[K[]]



SIMD Implementations

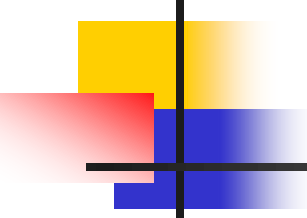
❑ Implementations:

- MMX (1996): 64-bit packed integers ($8 \times i8$, $4 \times i16$, $2 \times i32$)
- SSE (1999): 128-bit XMM, $4 \times f32$
- SSE2 (2001): adds $2 \times f64$ + full 128-bit integers ($16 \times i8$, $8 \times i16$, $4 \times i32$, $2 \times i64$)
- AVX (2011): 256-bit YMM, $8 \times f32$, $4 \times f64$
- FMA3 (2013); AVX2 (2013): 256-bit integers + gathers
- AVX-512 (2016+): 512-bit ZMM, mask regs k0–k7, gather/scatter; subsets F/DQ/BW/VL/VNNI/BF16
- ARM: NEON (128-bit), SVE/SVE2 (vector-length agnostic)



Example SIMD Code

	L.D	F0,a	;load scalar a
	MOV	F1, F0	;copy a into F1 for SIMD MUL
	MOV	F2, F0	;copy a into F2 for SIMD MUL
	MOV	F3, F0	;copy a into F3 for SIMD MUL
	DADDIU	R4,Rx,#512	;last address to load
Loop:	L.4D	F4,0[Rx]	;load X[i], X[i+1], X[i+2], X[i+3]
	MUL.4D	F4,F4,F0	;a×X[i],a×X[i+1],a×X[i+2],a×X[i+3]
	L.4D	F8,0[Ry]	;load Y[i], Y[i+1], Y[i+2], Y[i+3]
	ADD.4D	F8,F8,F4	;a×X[i]+Y[i], ..., a×X[i+3]+Y[i+3]
	S.4D	0[Ry],F8	;store into Y[i], Y[i+1], Y[i+2], Y[i+3]
	DADDIU	Rx,Rx,#32	;increment index to X
	DADDIU	Ry,Ry,#32	;increment index to Y
	DSUBU	R20,R4,Rx	;compute bound
	BNEZ	R20,Loop	;check if done



Speed Up Computing

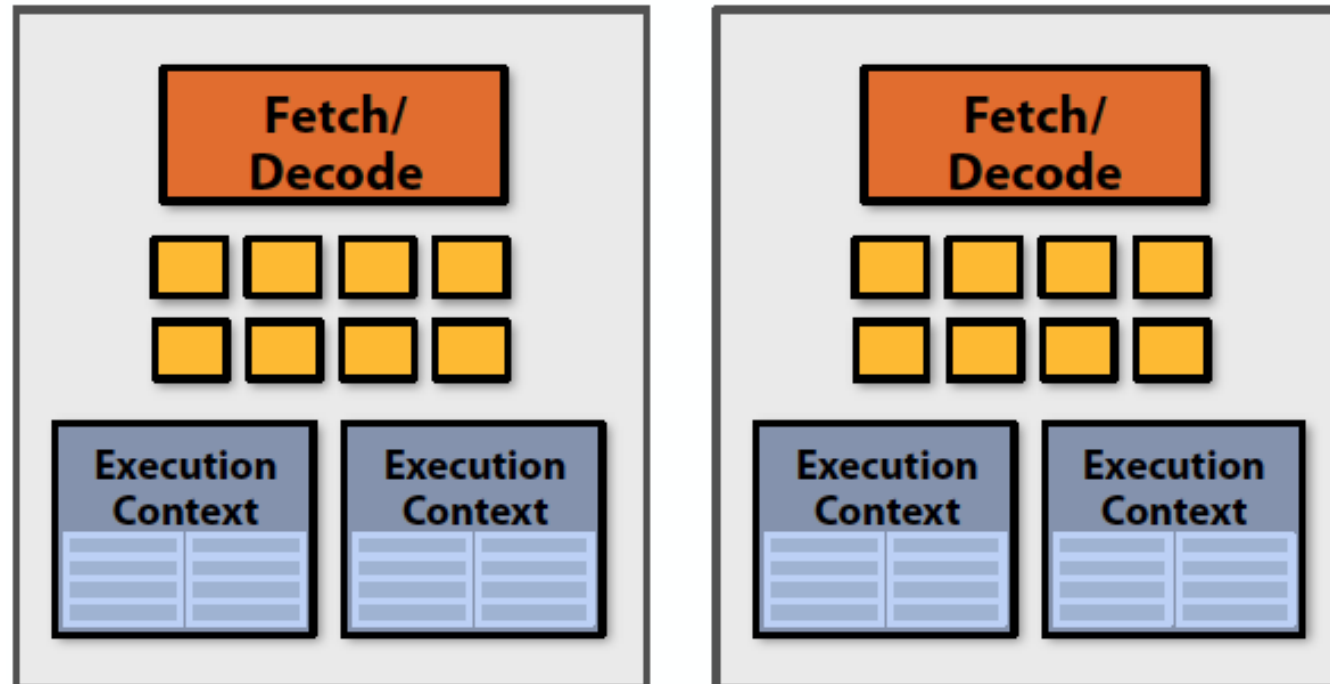
So far, we have discussed options to accelerate executing instructions via

1. Exploiting ILP through pipelining, branch prediction, hardware speculation, dynamic scheduling with register renaming, and superscalar
2. Exploiting DLP through SIMD

Any other options at the microarchitecture level?

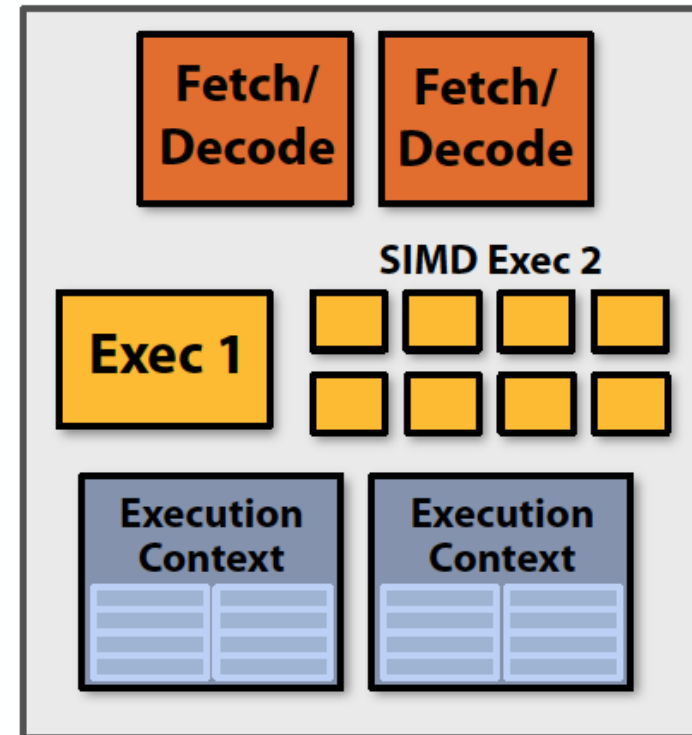
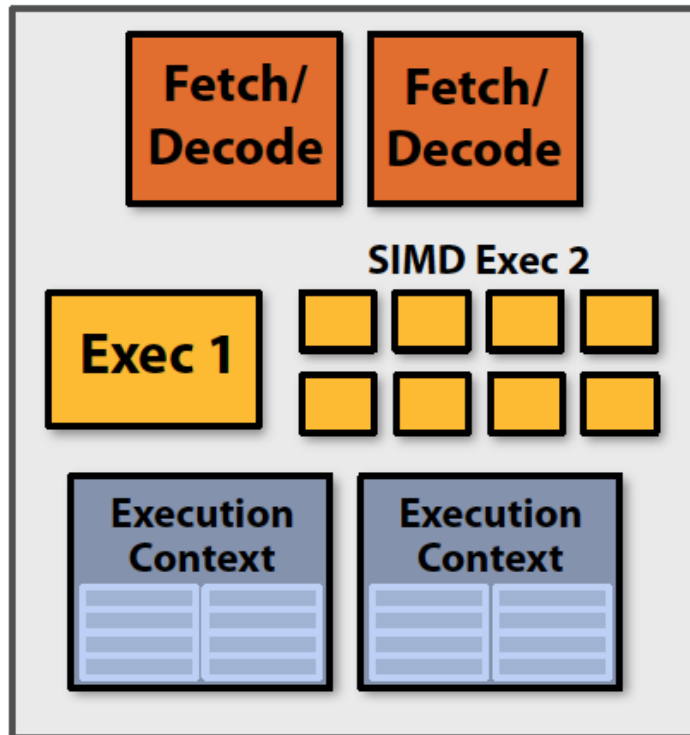
Simultaneous Multithreading (SMT)

- ❑ Improve the efficiency of superscalar CPUs with hardware multithreading → enable multiple independent threads of execution in order to better utilize the hardware resources
- ❑ Hide latency of loading data for one thread by executing instructions from other thread



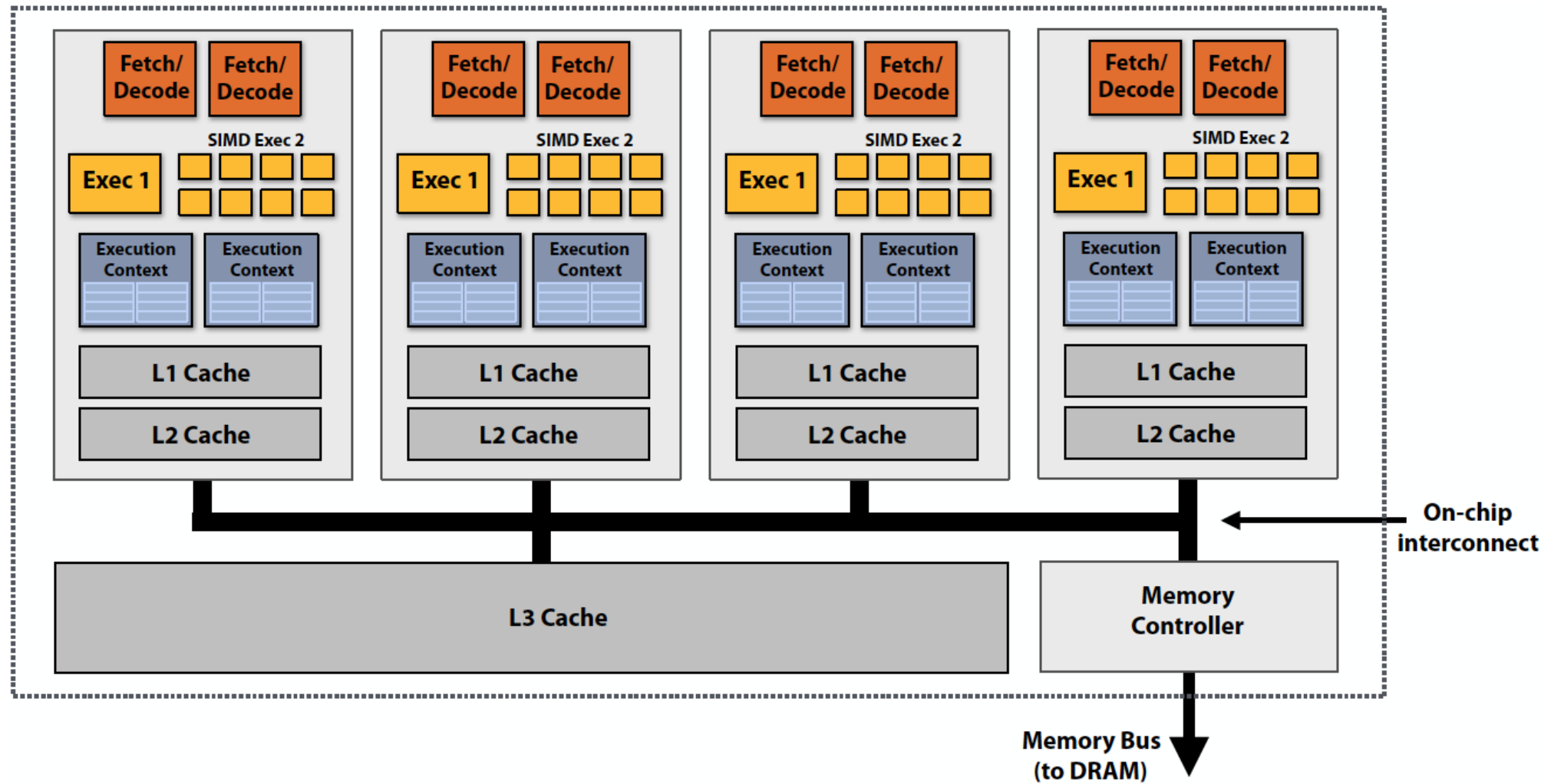
Simultaneous Multithreading (SMT)

- ❑ SMT + superscalar → higher hardware utilization efficiency
- ❑ E.g., up to two instructions per clock (one SIMD instruction + one scalar instruction)



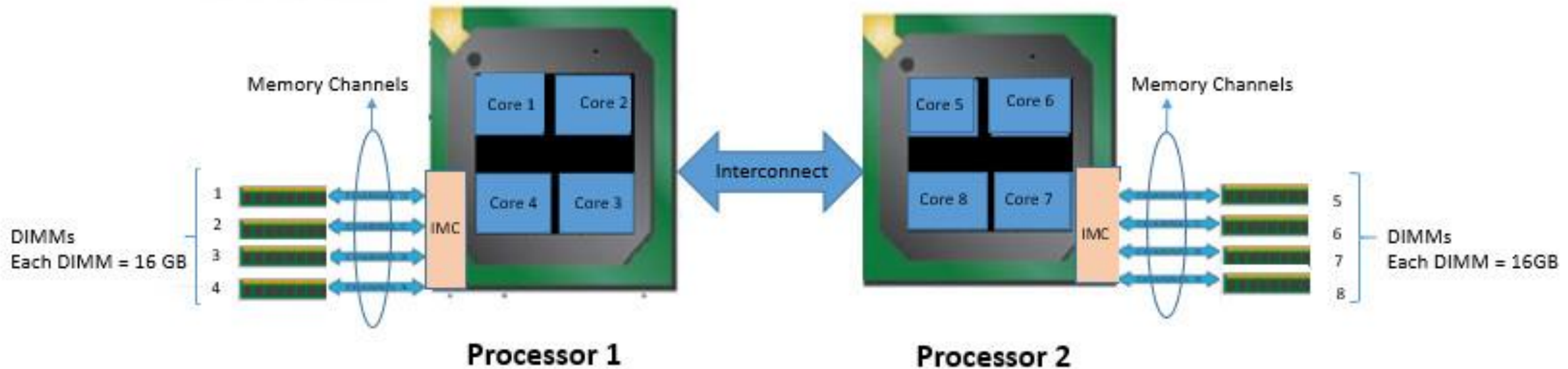
Multi-core + Superscalar + SMT

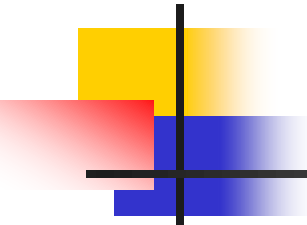
Example: Four cores, two-way SMT per core (max eight threads active on chip at once), up to two instructions per clock per core (one of those instructions is 8-wide SIMD)



NUMA: Non-uniform Memory Access

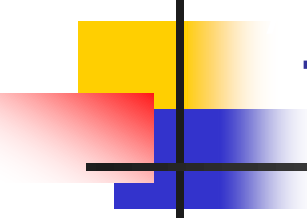
Multiple CPUs in one server





Multi-core + Superscalar + SMT + NUMA

- ❑ Now let's quickly look at the SW side
- ❑ Goal: Improve the core utilization to speed up computing
- ❑ Three levers: pin threads, place memory, pick granularity.
- ❑ Rule of thumb: Prefer 1 HW thread/core before SMT siblings
- ❑ Memory placement: heuristic, interleaving, NUMA-unaware (**which one is better?**)
- ❑ Pinning
 - CPU affinity: fix a thread to core(s) so it doesn't bounce across cores/sockets
 - Memory affinity: allocate/place pages on the same NUMA node as the thread
 - **Any pitfalls?**
- ❑ Work-chunk partitioning
 - OS schedules kernel threads on CPUs
 - user-level runtime schedules the *work* onto those threads (partition the work appropriately)



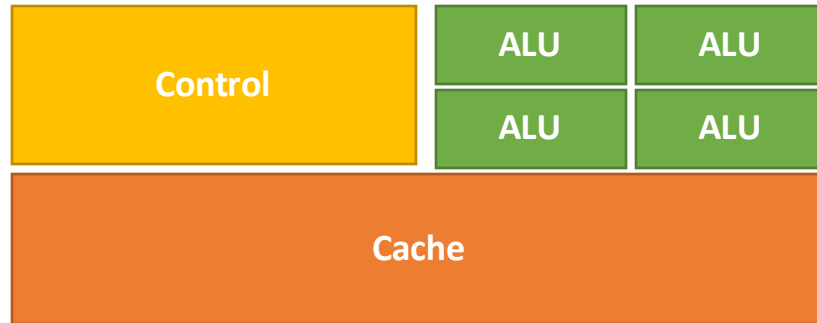
The Limit of Parallel Computing

- ❑ In general, if an application has:
 - t sequential execution time
 - p fraction of execution that is parallelizable
 - s speedup achieved on the parallelizable part

- ❑ What is the overall speedup of the application? Amdahl's Law

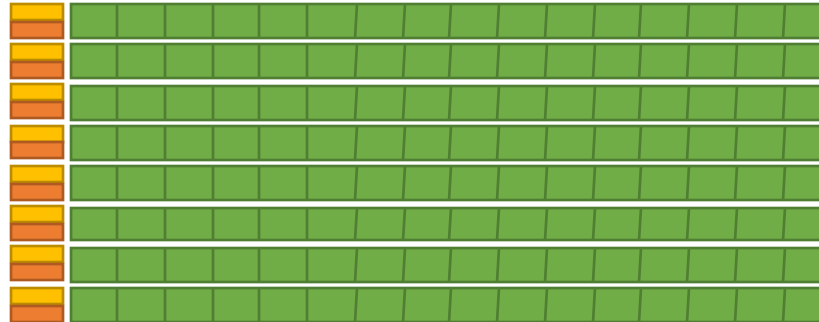
Computing

CPU: Latency-Oriented Computing

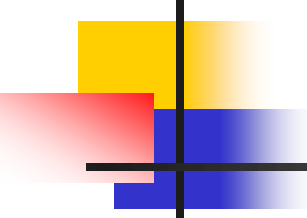


- A few powerful **ALUs**
 - Reduced operation latency
- Large **caches**
 - Convert long latency memory accesses to short latency cache accesses
- Sophisticated **control**
 - Branch prediction to reduce control hazards
 - Data forwarding to reduce data hazards
- Modest multithreading to hide short latency
- High clock frequency

GPU: Throughput-Oriented Computing



- Many small **ALUs**
 - Long latency, high throughput
 - Heavily pipelined for further throughput
- Small **caches**
 - More area dedicated to computation
- Simple **control**
 - More area dedicated to computation
- Massive number of threads to hide very high latency!
- Moderate clock frequency



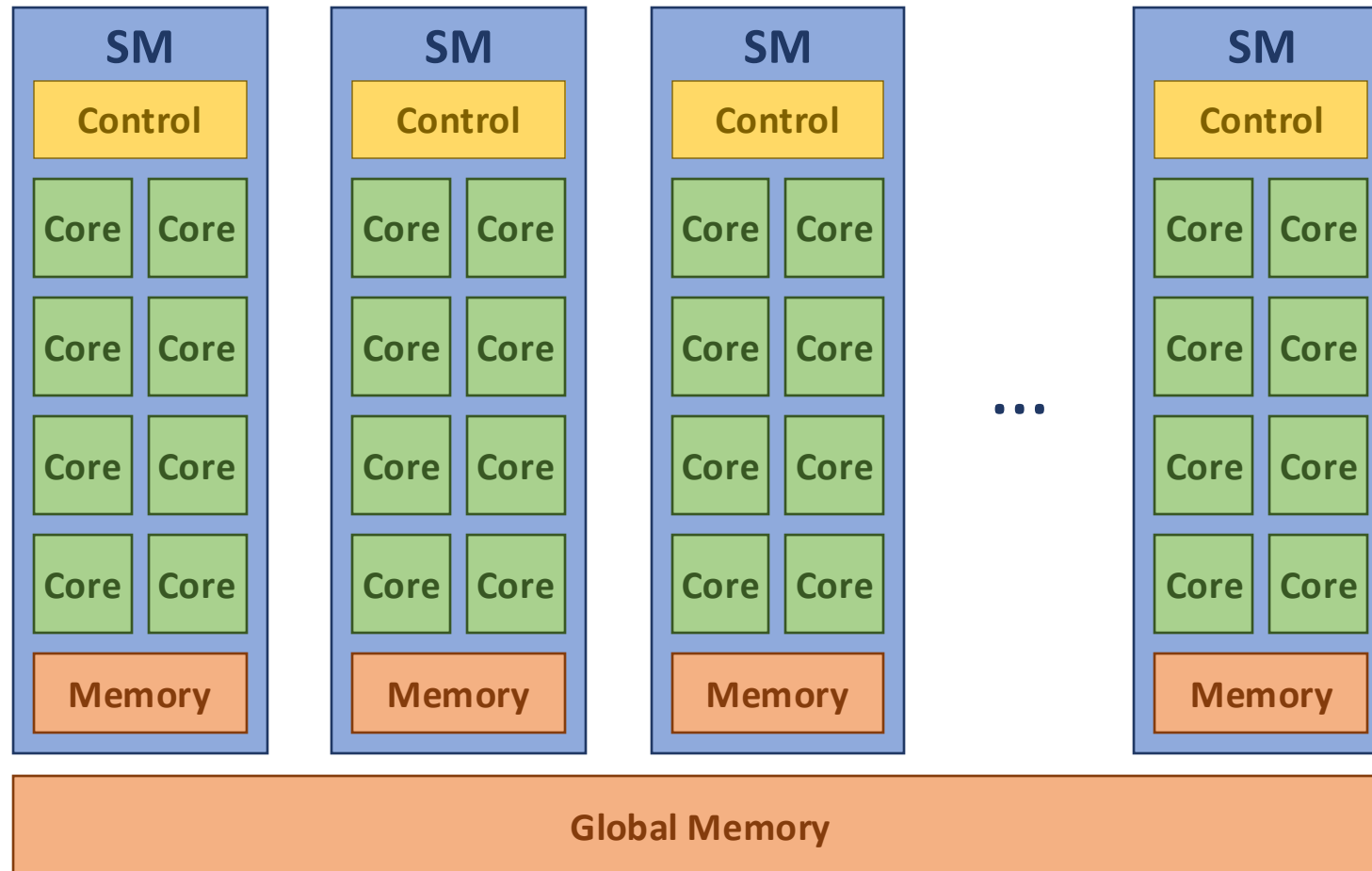
Beyond CPU: GPU

- ❑ Given the hardware invested to do graphics well, how can we apply GPU to improve the performance of a wider range of applications?
 - Heterogeneous execution model
 - ❖ CPU is the *host*, GPU is the *device* (why not running OS on GPU?)
 - Develop a C-like programming language for GPU
 - Programming model is “Single Instruction Multiple Thread”
- ❑ GPU key ideas:
 - Use many slimmed down cores to run in parallel
 - Pack many scalar ALUs together and have them share an instruction stream across multiple threads (SIMT)
 - Avoid latency stalls by interleaving execution of many groups of threads
 - ❖ When one group stalls, work on another group, instead of complex out-of-order

GPU Architecture

A GPU consists of multiple **Streaming Multiprocessor (SMs)**, each consisting of multiple **cores** with shared **control** and **memory**

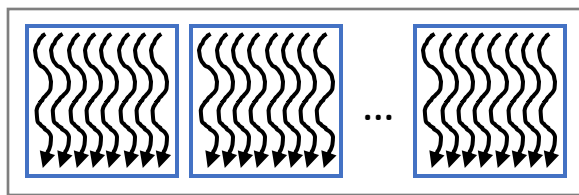
(e.g., a Volta V100 GPU has 80 SMs with 64 cores each which totals 5120 cores)



GPU Architecture

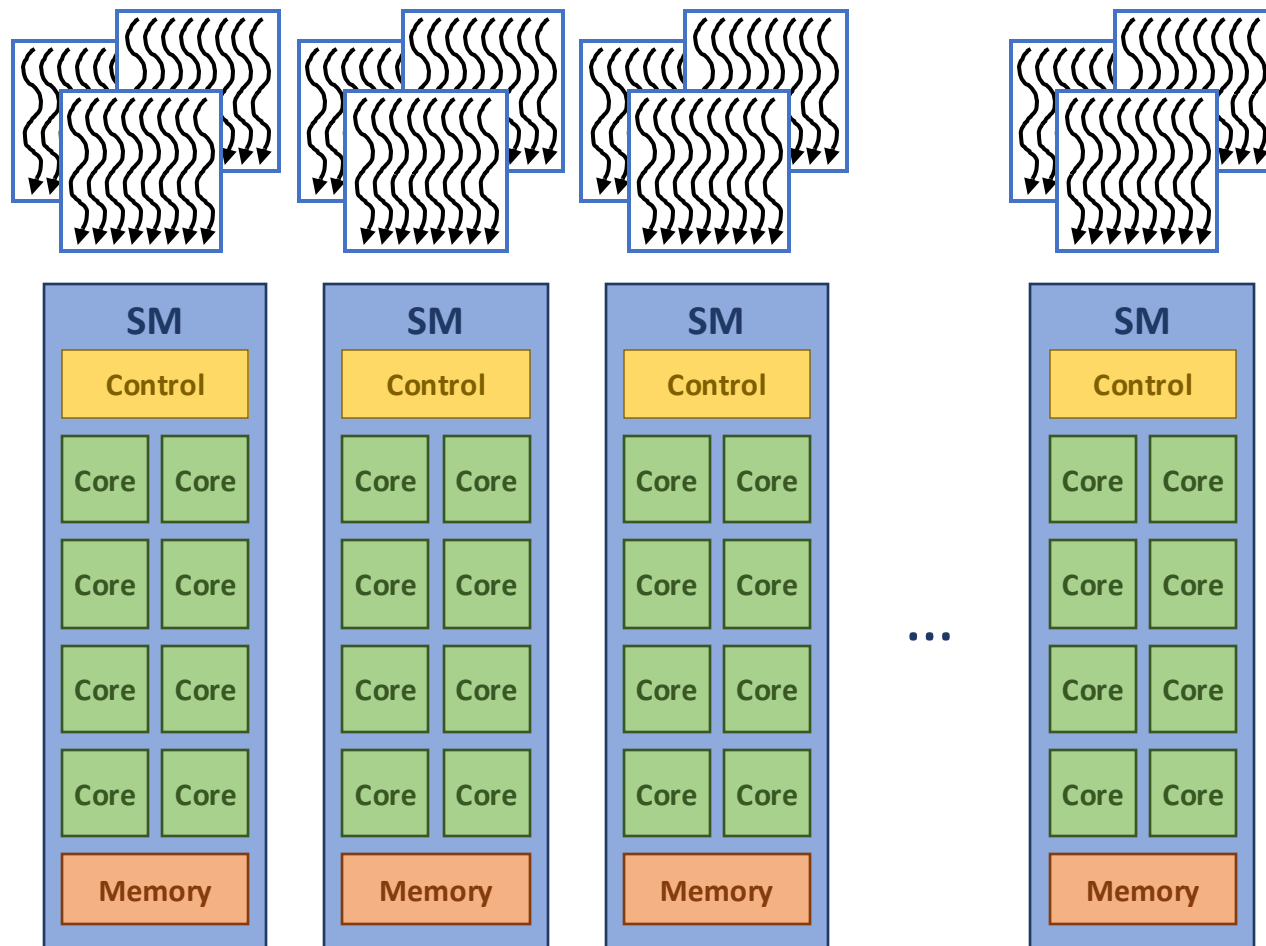
Threads/blocks require resources to execute (e.g., registers, memory, ...) so SMs can accommodate a limited number of threads/blocks at once...

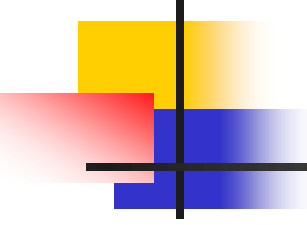
... The remaining blocks wait for other blocks to finish before they can be assigned to an SM.



Waiting

Threads are assigned to SMs at block granularity
(all threads in a block are assigned to the same SM)



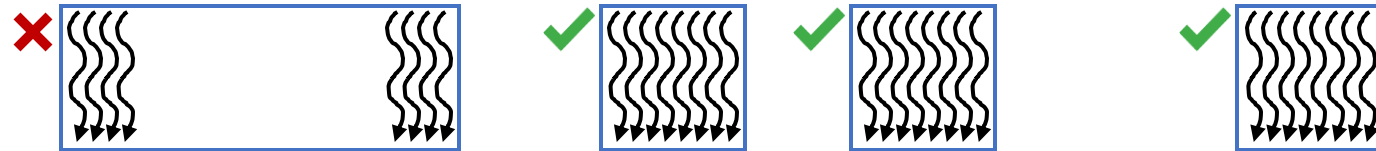


Synchronization

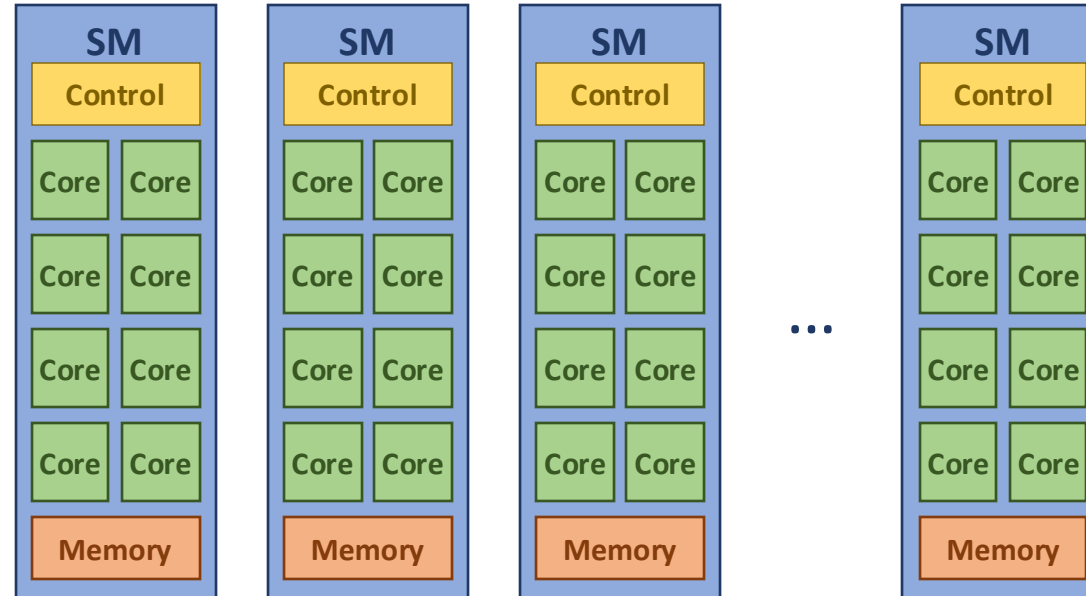
- ❑ Threads in the same block can collaborate in ways that threads in different blocks cannot:
 - Barrier synchronization: `__syncthreads()`
 - ❖ Wait for all threads in the block to reach the barrier before any thread can proceed
 - Shared memory
 - ❖ Access a fast memory that only threads in the same block can access

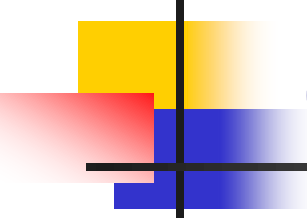
Scheduling

- ❑ All threads in a block are assigned to an SM simultaneously
 - A block cannot be assigned to an SM until it secures enough resources for all its threads to execute



Otherwise, if some threads reach a barrier and others cannot execute, the system could deadlock





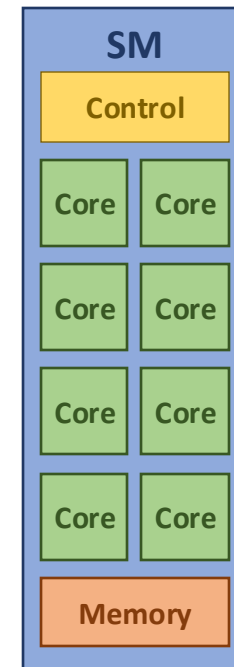
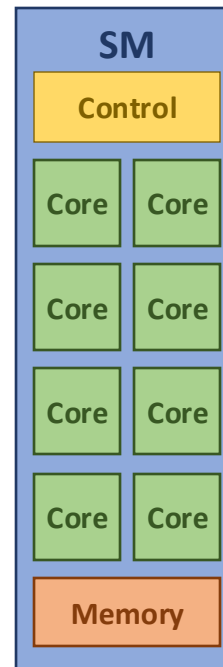
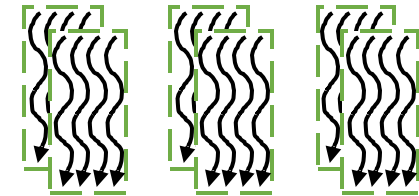
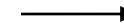
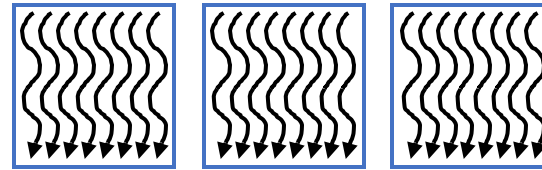
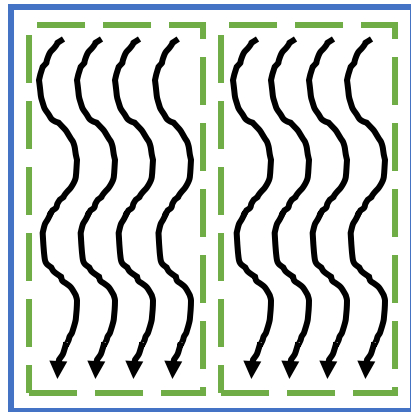
GPU Architecture

- ❑ Threads in different blocks do not synchronize
 - Allows blocks to execute in any order
 - Allows blocks to execute both in parallel with each other or sequentially with respect to each other
- ❑ Enables **transparent scalability**
 - Same code can run on different devices with different amounts of hardware parallelism
 - ❖ Execute blocks sequentially if device has few SMs
 - ❖ Execute blocks in parallel if device has many SMs
- ❑ Programmers should not write code that tries to synchronize across blocks
 - Deadlock occurs if the blocks are not scheduled simultaneously
 - Note: *cooperative groups* is an advanced feature of CUDA that allows synchronization across blocks by limiting the number of blocks to guarantee that all blocks are executing simultaneously

Thread Scheduling

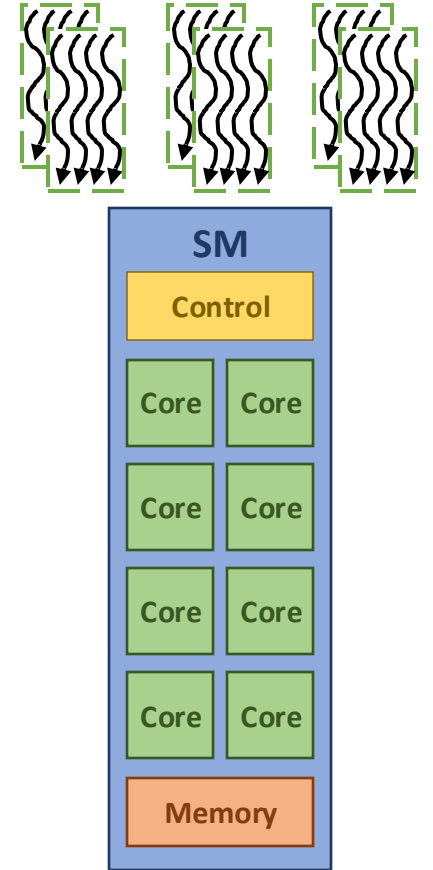
- ❑ Threads assigned to an SM run concurrently
 - The SM has a scheduler that manages their execution

Blocks assigned to an SM are further divided into **warps** which are the unit of scheduling



Warps

- ❑ Warps are the unit of scheduling in an SM
 - The size of warps is device-specific, but has always been 32 threads to date
- ❑ Threads in a warp are scheduled together and executed following the **SIMD** model
 - One instruction is fetched and executed by all the threads in the warp, each processing different data



Control Diverge Example

Example 1: Threads take different branches of a conditional

```
if (threadIdx.x < 24) {
```

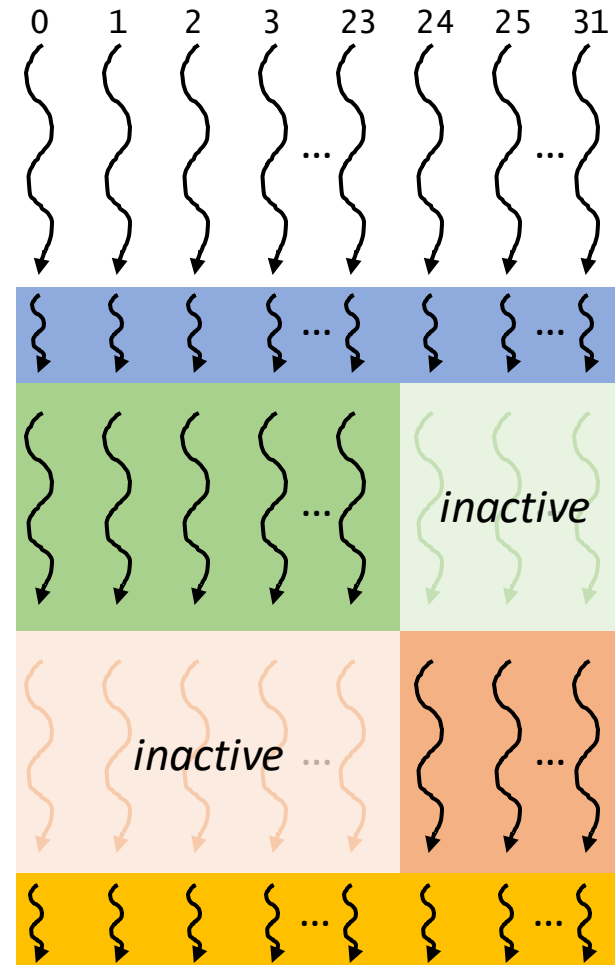
A

```
} else {
```

B

```
}
```

C



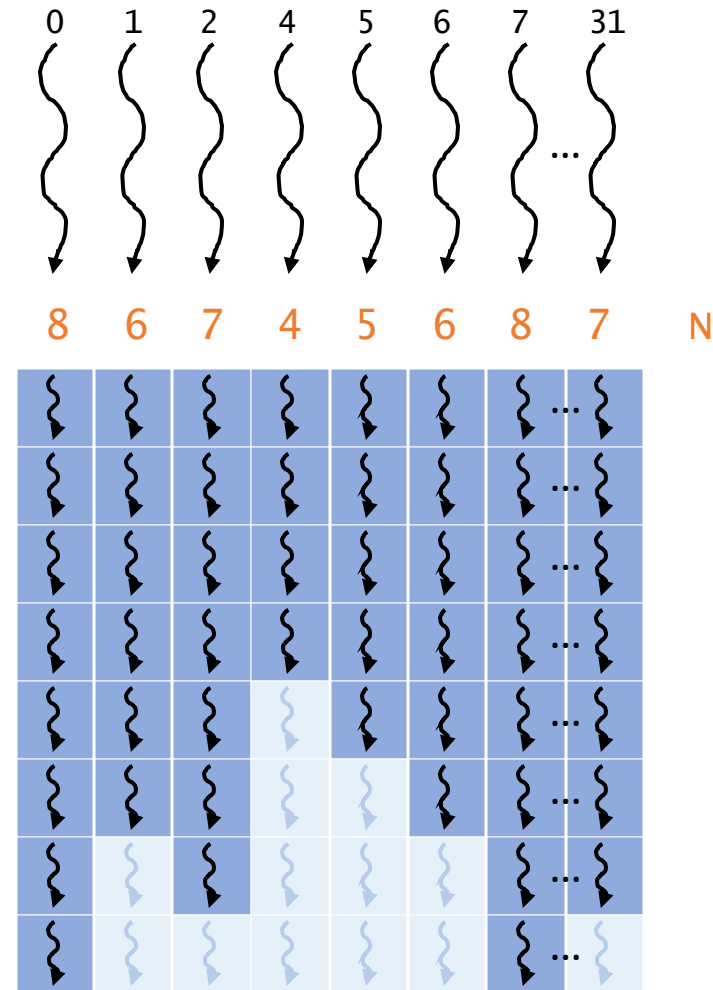
Control Diverge Example

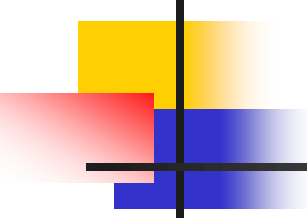
Example 2: Threads execute a different number of loop iterations

```
N = a[threadIdx.x];  
for(i = 0; i < N; ++i) {
```

A

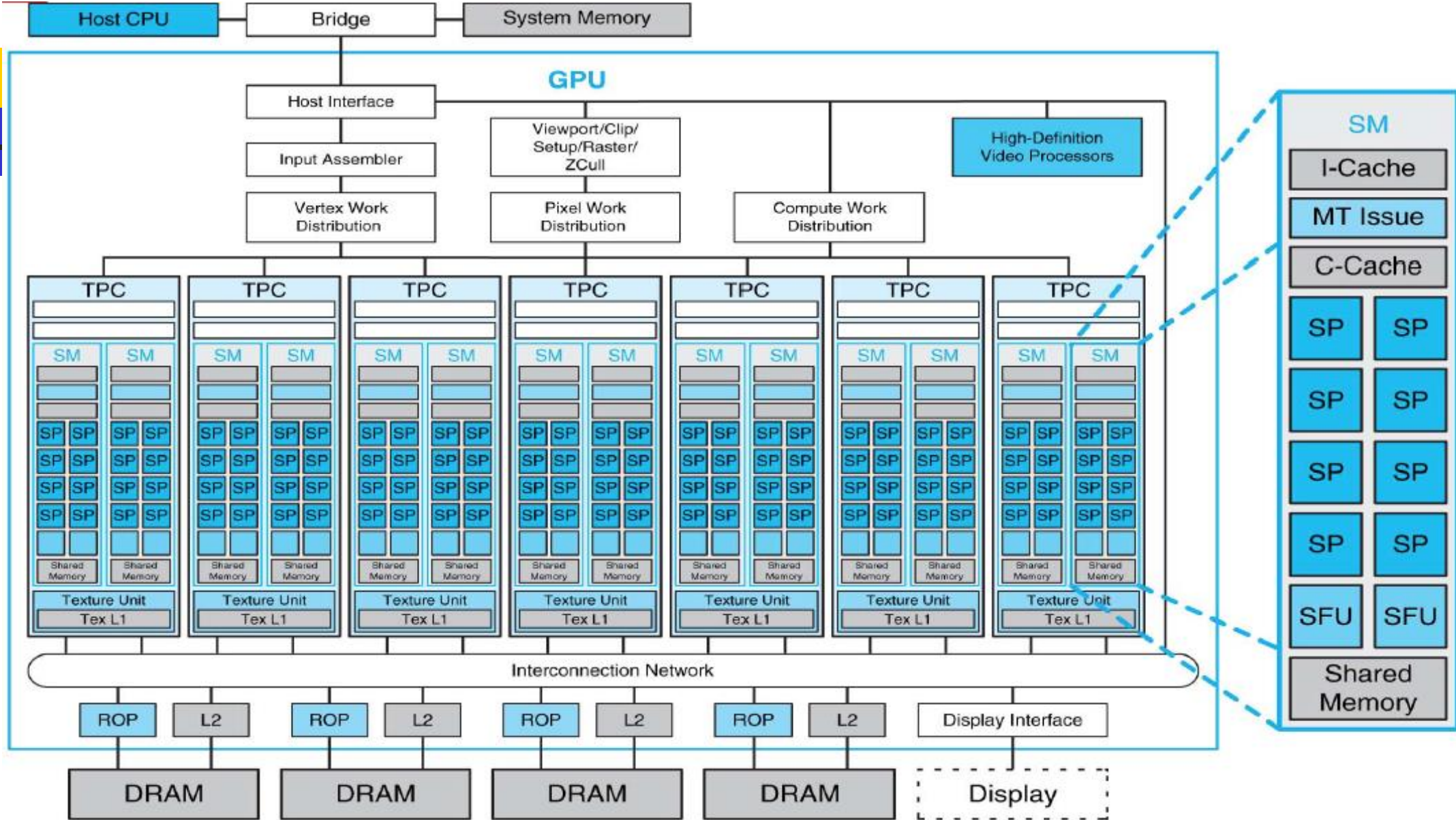
```
}
```



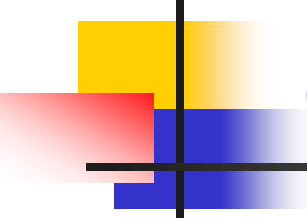


Latency Hiding

- ❑ When a warp needs to wait for a high latency operation, another warp that is ready is selected and scheduled for execution
- ❑ Many warps are needed to ensure sufficient work available to hide long latency operations
 - i.e., there is high chance of finding a warp that is ready
- ❑ For this reason, an SM typically supports many more threads than the number of its cores
- ❑ The **occupancy** of an SM refers to the ratio of the warps or threads active on the SM to the maximum allowed
- ❑ In general, maximizing occupancy is desirable because it improves latency hiding
 - Common case, but possible to have cases where lower occupancy is desirable (**how?**)

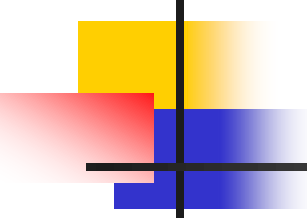


SP: Scalar processor 'CUDA core', execute one thread; SM (stream multiprocessor) consists of many SPs (e.g., 32, 64);
 SFU (special function unit): transcendental instructions e.g., sin, cos, square root;
 Local shared memory: Shared among SPs in one SM (e.g., 64KB); 30 SMs in GT200, 14 SMs in Fermi



GPU Programming

- ❑ OpenGL (Open Graphics Library): cross-platform API for rendering 2D/3D graphics
- ❑ CUDA (Compute Unified Device Architecture): parallel GPU programming API created by NVIDIA
 - CUDA is NOT a language, it is an API
 - Before CUDA, one had to repurpose graphics-specific APIs for non-graphics work
- ❑ A rich set of CUDA API libraries, e.g.,
 - cuBLAS Library: an implementation of BLAS (Basic Linear Algebra Subprograms)
 - cuTENSOR Library: a tensor linear algebra library for deep learning, computer vision, etc.
 - RAPIDS libraries suite: target at GPU-based data science and analytics, written in Python, and built on Apache Arrow



Speed Up Computing

Now we have discussed all the major microarchitecture design concepts for speeding up computing

1. Exploiting ILP through pipelining, branch prediction, hardware speculation, dynamic scheduling with register renaming, and superscalar
2. Exploiting data-level parallelism through SIMD
3. Exploiting task-level parallelism through multi-threading, GPU

Any other options at the microarchitecture level?

Modern server CPUs contain $\sim 100B$ transistors and $\sim 600MB$ cache: what is its implication?

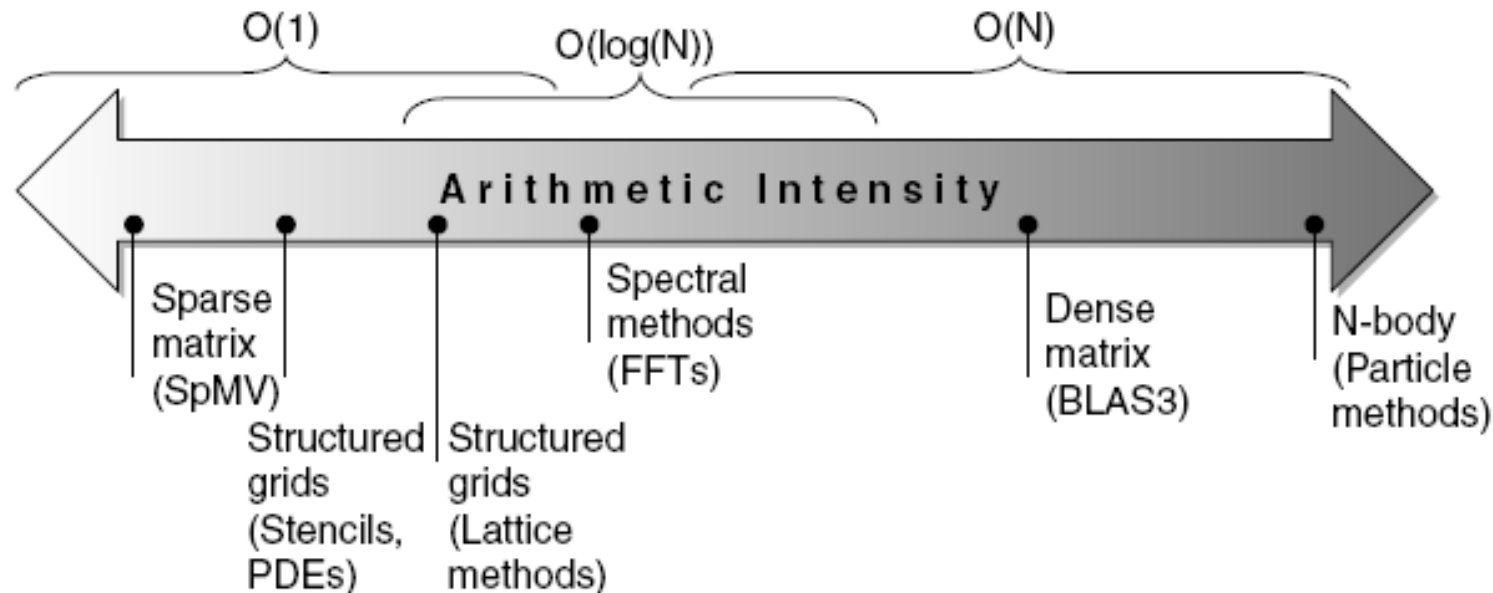
Roofline Performance Model

□ Basic idea:

- Plot peak floating-point throughput as a function of arithmetic intensity
- Ties together floating-point performance and memory performance for a target machine

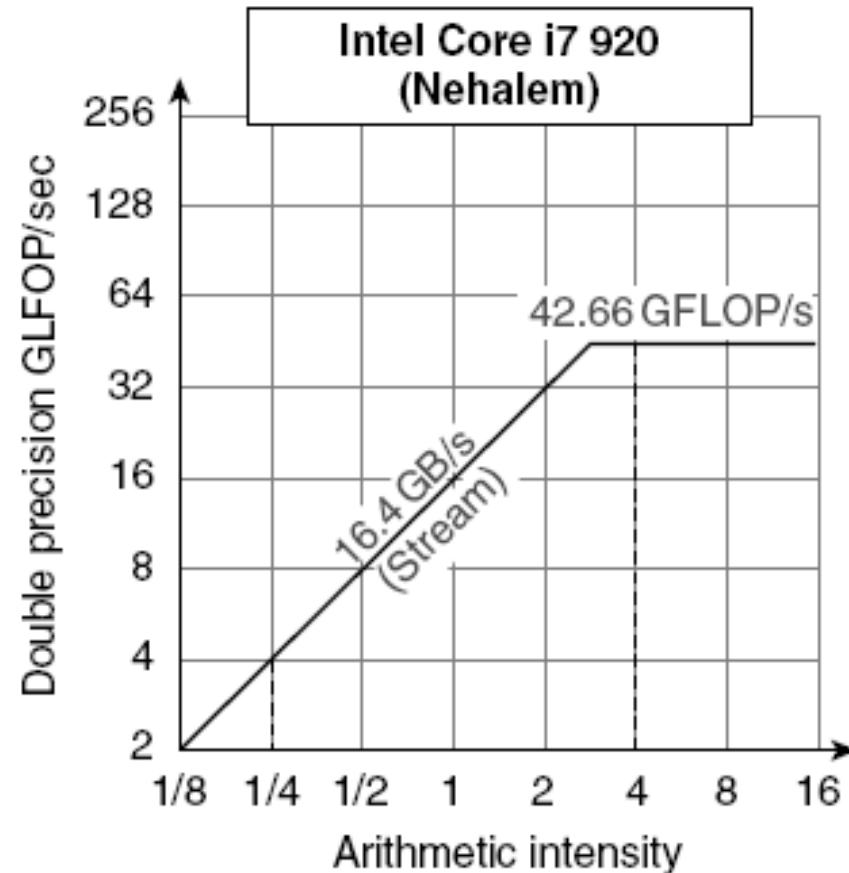
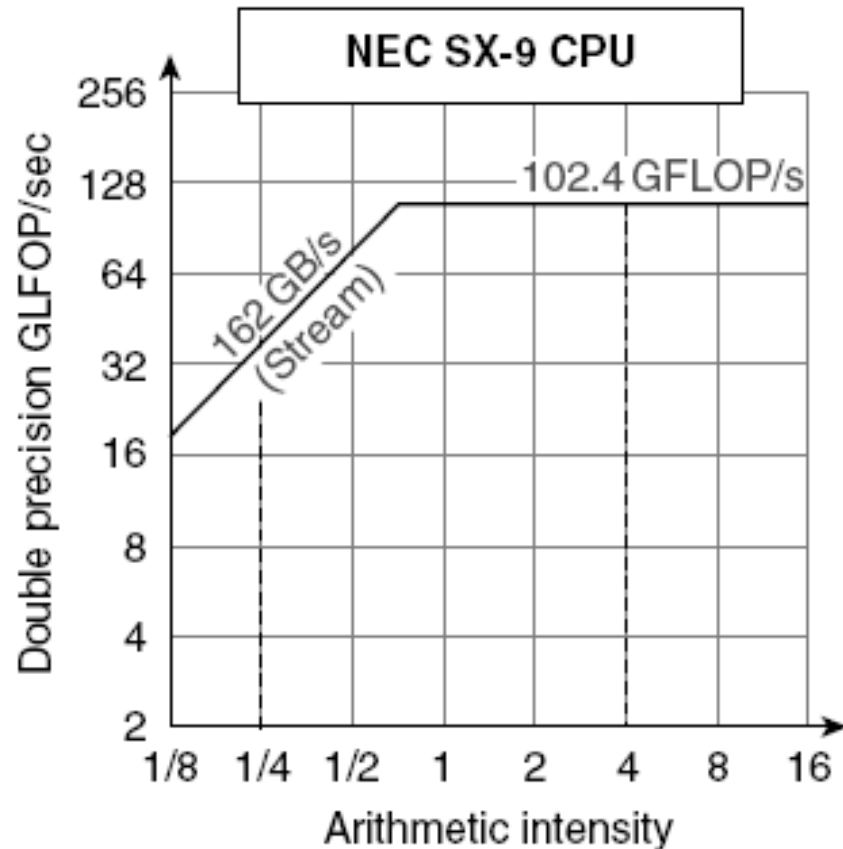
□ Arithmetic intensity

- Floating-point operations per byte read



Examples

Attainable GFLOPs/sec $\text{Min}(\text{Peak Memory BW} \times \text{Arithmetic Intensity}, \text{Peak Floating Point Perf.})$



Speed Up Computing

A simple logic

Speed up computing → increase computing resource utilization → closer to compute-bound region

